SAVITRIBAI PHULE PUNE UNIVERSITY

S.Y.B.Sc. (Computer Science) • NEP-2020 • Semester III Practical Examination

Lab Course CS-203-MJ-P
Data Structures & DBMS

Ultimate Easiest Slips Master Guide (19 High-Yield Slips • Memory Tricks, Simplified Logic & High-Scoring Viva)

30	3 Hrs	13 + 12	5
MAX MARKS	DURATION	DS + DBMS MARKS	VIVA MARKS

🎯 4 Golden Tactical Rules to Score 30/30 in 45 Minutes

1. The Array/Static Shortcut (DS): Always choose the array or static stack/queue option over Linked Lists or File Handling whenever offered. Array code runs in 20-30 lines and eliminates runtime segfaults!

2. The "Any 3" Rule (DBMS Part B): The exam paper explicitly instructs *"Execute the following queries in PostgreSQL (any 3) [6 marks]"*. Do NOT waste time solving all 5 queries. Write the 3 simplest queries first to lock in full marks.

3. Lean Data Inserts (DBMS Part A): Insert only 3 to 4 sample rows per table. Examiners only check whether your tables exist, constraints work, and queries return clean output.

4. Guaranteed Viva Score (5 Marks): Viva questions are 90% predictable! Review the Top 30 High-Yield Viva Questions & Answers in Part IV of this guide.

Curated for Rapid Exam Preparation • Arranged Strictly by Difficulty Level • Memory Anchors Included

🖥️ PostgreSQL Practical Lab Setup & Terminal Guide

Follow this exact terminal workflow in your college Linux/Ubuntu lab. This matches the lab environment 100%.

```
student@lab-pc: ~ (PostgreSQL Ubuntu Terminal Walkthrough)

# STEP 1: Launch PostgreSQL CLI from Ubuntu Terminal
student@lab-pc:~$ sudo -u postgres psql
psql (16.2 (Ubuntu 16.2-1ubuntu4))

# STEP 2: Create an Isolated Database for Your Slip & Connect
postgres=# CREATE DATABASE slip01_db;
postgres=# \c slip01_db
You are now connected to database "slip01_db" as user "postgres".

# STEP 3: Create Tables and Insert Data (Paste Table DDL & INSERTS)
slip01_db=# CREATE TABLE Room (
    roomno INT PRIMARY KEY,
    room_name VARCHAR(50),
    room_type VARCHAR(10) CHECK (room_type IN ('AC', 'NonAC')),
    charges NUMERIC(10,2)
);
CREATE TABLE

# STEP 4: Verify Tables and Schema Structure
slip01_db=# \dt
             List of relations
 Schema | Name  | Type  |  Owner
--------+-------+-------+----------
 public | guest | table | postgres
 public | room  | table | postgres
(2 rows)

slip01_db=# \d Room
                          Table "public.room"
  Column   |         Type          | Collation | Nullable | Default
-----------+-----------------------+-----------+----------+---------
 roomno    | integer               |           | not null |
 room_name | character varying(50) |           |          |
 charges   | numeric(10,2)         |           |          |
Indexes: "room_pkey" PRIMARY KEY, btree (roomno)

# STEP 5: Execute Part B Queries & Show Output to Examiner
slip01_db=# SELECT * FROM Room WHERE charges BETWEEN 5000 AND 10000;
 roomno |  room_name  | room_type | charges
--------+-------------+-----------+---------
    101 | Deluxe King | AC        | 6500.00
(1 row)

# STEP 6: Exit when examiner has checked
slip01_db=# \q
student@lab-pc:~$
```

⚡ The 6 Essential psql Meta-Commands (Examiner Favorite):

Command	What It Does	When to Use in Exam
`\c dbname`	Connects to a specific database	Immediately after `CREATE DATABASE`
`\dt`	Lists all created tables	Show this to examiner to prove Part A is done
`\d tablename`	Describes table schema & constraints	Show PRIMARY KEY, FOREIGN KEY, CHECK constraints
`\i script.sql`	Executes external .sql file script	If you saved your queries in a file
`\dv`	Lists all views in current database	Show this after creating view for Part C
`\q`	Quits psql back to Linux bash shell	When examiner has signed your slip

📚 PostgreSQL Data Types & Constraints (Write Any Table Yourself)

Everything you need to write table definitions from scratch on exam day. Pick the right data type and constraint in 5 seconds!

1. Text Types: VARCHAR vs CHAR vs TEXT

> 💡 **Formula: Always default to VARCHAR(50) for names, cities, strings!**

• **VARCHAR(n):** Variable-length string up to *n* characters. Only consumes disk space for actual letters typed + 1 byte! Perfect for: `name VARCHAR(50)`, `city VARCHAR(30)`, `designation VARCHAR(40)`.
• **CHAR(n):** Fixed-length string. Always pads with blank spaces to occupy exact *n* characters. Best for fixed codes: `state_code CHAR(2)`, `gender CHAR(1)`.
• **TEXT:** Variable length without any maximum limit. Use for long descriptions: `symptoms TEXT`.

2. Numeric Types: INT vs NUMERIC

> 💡 **Formula: INT for IDs & Counts | NUMERIC(10,2) for Money & Salary!**

• **INT / INTEGER:** Whole numbers (-2 billion to +2 billion). Best for: `id INT`, `age INT`, `capacity INT`, `pincode INT`.
• **NUMERIC(p, s) / DECIMAL:** Exact decimal representation! *p* = total digits, *s* = digits after decimal point.
Example: `NUMERIC(10, 2)` can store up to 99999999.99.
Use for: `salary NUMERIC(10,2)`, `charges NUMERIC(10,2)`, `balance NUMERIC(12,2)`.

3. Date & Boolean Types

> 💡 **Formula: DATE values must always be inside single quotes 'YYYY-MM-DD'!**

• **DATE:** Calendar date. Format: `'YYYY-MM-DD'` (e.g. `'2025-04-01'` or `'1998-07-15'`).
Use for: `birthdate DATE`, `start_date DATE`, `date_of_duty DATE`.
• **BOOLEAN:** Stores TRUE or FALSE (or `'t'` / `'f'`).
Use for binary flags: `is_active BOOLEAN`.

4. The 5 Golden Table Constraints

> 💡 **Formula: PK enforces Unique+Not-Null; FK links to parent; CHECK restricts!**

• **PRIMARY KEY:** Uniquely identifies row, cannot be NULL: `eno INT PRIMARY KEY`.
• **FOREIGN KEY:** Links to parent table's PK: `dno INT REFERENCES Department(dno)`.
• **CHECK:** Enforces domain rules: `CHECK (room_type IN ('AC', 'NonAC'))` or `CHECK (salary > 5000)`.
• **UNIQUE:** Prevents duplicate values: `license_no VARCHAR(20) UNIQUE`.
• **NOT NULL:** Mandatory column: `name VARCHAR(50) NOT NULL`.

```sql
-- MASTER TEMPLATE: How to Write ANY Table Definition (Copy this pattern in lab):
CREATE TABLE Employee (
    eno INT PRIMARY KEY, -- Primary Key (Unique + Not Null)
    ename VARCHAR(50) NOT NULL, -- Text name up to 50 chars
    salary NUMERIC(10, 2) CHECK (salary > 5000), -- Decimal number with validation rule
    hire_date DATE, -- Date formatted as 'YYYY-MM-DD'
    dno INT REFERENCES Department(dno) -- Foreign Key pointing to Department
);
```

⚡ The 6 Universal Master Code Skeletons (Memorize These 6!)

90% of the Data Structures practical exam boils down to these 6 exact patterns. Memorize the 4-line core logic and you can solve ANY slip:

1. Linear Search

Slips: 3, 9, 16

> 💡 **Formula: Single loop 0 to n-1. If arr[i]==x, save pos and break!**

```
int pos = -1; for (int i = 0; i < n; i++) { if (arr[i] == x) { pos = i; break; } } if (pos != -1) printf("Found at %d\n", pos); else printf("Not found\n");
```

2. Binary Search

Slips: 7, 13

> 💡 **Formula: low=0, high=n-1, mid=(l+h)/2. Halve the range!**

```
int low = 0, high = n - 1, pos = -1; while (low <= high) { int mid = (low + high) / 2; if (arr[mid] == x) { pos = mid; break; } else if (x < arr[mid]) high = mid - 1; else low = mid + 1; }
```

3. Bubble Sort

Slips: 8, 18

> 💡 **Formula: Compare adjacent neighbors. Heavy bubbles sink!**

```
for (int i = 0; i < n - 1; i++) { for (int j = 0; j < n - 1 - i; j++) { if (arr[j] > arr[j + 1]) { int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t; } } }
```

4. Selection Sort

Slips: 6, 12

> 💡 **Formula: Find smallest index (min_idx), single swap per round.**

```
for (int i = 0; i < n - 1; i++) { int min_idx = i; for (int j = i + 1; j < n; j++) { if (arr[j] < arr[min_idx]) min_idx = j; } int t = arr[min_idx]; arr[min_idx] = arr[i]; arr[i] = t; }
```

5. Insertion Sort

Slips: 1, 15

> 💡 **Formula: Pick card key. Slide bigger cards right, drop key!**

```
for (int i = 1; i < n; i++) { int key = arr[i], j = i - 1; while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; // Shift right j--; } arr[j + 1] = key; // Insert }
```

6. Static Stack & Queue

Slips: 1, 2, 4, 10, 11, 20, 21

> 💡 **Formula: Stack: s[++top]=x, s[top--]. Queue: q[++rear]=x, q[front++].**

```
// STACK: top starts at -1 void push(int x) { s[++top] = x; } int pop() { return s[top--]; } // QUEUE: front=-1, rear=-1 void insert(int x) { if(front==-1) front=0; q[++rear]=x; } void delete() { printf("%d\n", q[front++]); }
```

🗄️ **Universal PostgreSQL Query Cheat Formula:**
1. **Filter:** `SELECT * FROM table WHERE col = 'val';` | 2. **Pattern:** `WHERE name LIKE 'R%';` (starts) or `'%in%'` (contains) | 3. **Aggregate:** `SELECT AVG(col), COUNT(*) FROM table;` | 4. **Update:** `UPDATE table SET col = col * 1.15 WHERE cond;` | 5. **View:** `CREATE VIEW my_view AS SELECT ... FROM ...;`

📊 Master Difficulty Hierarchy Matrix (Slips Ranked 1 to 25)

#	Slip	Tier	Data Structures	DBMS
#1	S3	TIER 1	Linear Search	Person & Area
#2	S9	TIER 1	Linear Search	Person & Area
#3	S8	TIER 1	Bubble Sort	Student & Teacher
#4	S18	TIER 1	Bubble Sort	Student & Teacher
#5	S6	TIER 1	Selection Sort	Student & Teacher
#6	S12	TIER 1	Selection Sort	Sales_order & Client
#7	S1	TIER 1	Static Stack	Room & Guest
#8	S4	TIER 1	Reverse String using Static Stack	Policy & Customer
#9	S11	TIER 1	Reverse String using Static Stack	Emp & Project
#10	S7	TIER 2	Binary Search	Item & Supplier
#11	S13	TIER 2	Binary Search	Bus & Route
#12	S15	TIER 2	Insertion Sort	Game & Player
#13	S17	TIER 2	Static Stack	Property & Owner

#	Slip	Tier	Data Structures	DBMS
#14	S2	TIER 2	Static Queue	Employee & Department
#15	S10	TIER 2	Static Stack	Customer & Account
#16	S20	TIER 2	Static Stack	Customer & Account
#17	S21	TIER 2	Static Queue	Bus & Driver
#18	S23	TIER 2	Static Stack	Customer & Account
#19	S22	TIER 3	Singly Linked List	Employee & Department
#20	S24	TIER 2	Static Queue	Bus & Route
#21	S5	TIER 3	Singly Linked List	Doctor & Patient
#22	S14	TIER 3	Singly Linked List	Doctor & Patient
#23	S25	TIER 3	Postfix Evaluation using Stack	Employee & Project
#24	S19	TIER 3	Dynamic Queue	Order & Client
#25	S16	TIER 3	Linear Search on Cities	Student & Teacher

⚡ Numeric Quick Lookup (Slip 1 to 25)

Slip	Rank	Tier	Slip	Rank	Tier	Slip	Rank	Tier	Slip	Rank	Tier	Slip	Rank	Tier
S1	#7	TIER 1	S6	#5	TIER 1	S11	#9	TIER 1	S16	#25	TIER 3	S21	#17	TIER 2
S2	#14	TIER 2	S7	#10	TIER 2	S12	#6	TIER 1	S17	#13	TIER 2	S22	#19	TIER 3
S3	#1	TIER 1	S8	#3	TIER 1	S13	#11	TIER 2	S18	#4	TIER 1	S23	#18	TIER 2
S4	#8	TIER 1	S9	#2	TIER 1	S14	#22	TIER 3	S19	#24	TIER 3	S24	#20	TIER 3
S5	#21	TIER 3	S10	#15	TIER 2	S15	#12	TIER 2	S20	#16	TIER 2	S25	#23	TIER 3

TIER 1: SUPER EASY / RAPID MASTERY

The fastest, lowest-effort slips. Highly recommended for rapid revision tonight!

SLIP 3 [Rank #1 of 25]

TIER 1 ⏱️ 10-12 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Accept the array of n integers from user and sort the array in ascending order by using recursive Quick sort algorithm. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (primary keys are underlined).
Person (pnumber, pname, birthdate, income)
Area (aname, area_type,pincode)
The relationship is as follows: Person-Area: many-to-one. The 'area_type' can have values as either
'urban' or 'rural'.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL(any 3)
[6 marks]
i. List the details of all people whose name starts with the alphabet 'R'.
ii. List the names of all people whose birthday falls in the month of 'July'.
iii. Display the details of people in the sorted order of their income.
iv. Display the count of areas of 'urban' type.
v. Change the pincode of 'kalyaninagar' to 411036.
C. Write a query to list the names of people who live in 'Camp' area and have income less than at
least one person who lives in 'kalyaninagar' area.
[2 marks]
OR
C. Create a view to list the details of the person with second maximum income.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Linear Search is the most intuitive search method: imagine checking a row of closed doors one by one looking for a friend. You check door 0, door 1, door 2... If `arr[i] == x`, you've found your target! You save the index (`pos = i`) and immediately break out of the loop. If you reach the end of the array and `pos == -1`, it means the number was never in the array. Choosing this option completely avoids the tricky recursion and partitioning of Quick Sort.

🔹 **DBMS & PostgreSQL Logic:** Scenario: City Planning Registry. Citizens (Person) reside in specific geographical Areas (urban or rural). Area has `aname PRIMARY KEY` and a `CHECK (area_type IN ('urban', 'rural'))` constraint. Person references `aname`. For queries: Query (i) finds persons whose name begins with 'M' using `WHERE pname LIKE 'M%'`. Query (ii) transfers residents to a new area with `UPDATE`. Query (iii) counts rural residents with `SELECT COUNT(*) FROM Person JOIN Area ON Person.aname = Area.aname WHERE area_type = 'rural'`.

🧠 **Mental Formula & Memory Trick:** The Flashlight Loop: Check each door from 0 to n-1. When arr[i] == x, memorize the room number (pos = i) and instantly yell STOP (break)! If pos remains -1, it wasn't there.

💡 **Winning Exam Strategy:** Easiest question in the entire syllabus! Only 20 lines of simple loop logic. Completely avoids the recursion, partition logic, and stack overhead of Quick Sort.

💻 Section I: Data Structures (13 Marks) — Linear Search on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, x, pos = -1;

    printf("Enter size of random array: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0)); // Seed random generator

    // [STEP 1]: Generate and display random numbers
    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100; // Random numbers 0 to 99
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Enter value x to search: ");
    scanf("%d", &x);

    // [STEP 2]: The Flashlight Loop (Scan sequentially)
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            pos = i; // Target found!
            break;   // Stop searching immediately
        }
    }

    // [STEP 3]: Report findings
    if (pos != -1)
        printf("Element %d found at position %d (index %d)\n", x, pos + 1, pos);
    else
        printf("Element %d is NOT present in the array\n", x);

    return 0;
}
```

Expected Terminal Output:

```
Enter size of random array: 5
Random Array: 42 17 89 23 56
Enter value x to search: 89
Element 89 found at position 3 (index 2)
```

🗄️ Section II: Database Management Systems (12 Marks) — Person & Area (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Area (
    aname VARCHAR(50) PRIMARY KEY,
    area_type VARCHAR(10) CHECK (area_type IN ('urban', 'rural')),
    pincode INT
);

-- Create Person table
CREATE TABLE Person (


    pnumber INT PRIMARY KEY,
    pname VARCHAR(50),
    birthdate DATE,
    income NUMERIC(10,2),
    aname VARCHAR(50),
    FOREIGN KEY (aname) REFERENCES Area(aname)
);

-- Insert sample records into Area
INSERT INTO Area VALUES
('Camp', 'urban', 411001),
('Kalyaninagar', 'urban', 411014),
('Hadapsar', 'rural', 412308),
('ShivajiNagar', 'urban', 411005);

-- Insert sample records into Person
INSERT INTO Person VALUES
(1, 'Ravi', '1995-07-12', 45000, 'Camp'),
(2, 'Meena', '1998-03-25', 30000, 'Kalyaninagar'),
(3, 'Raj', '1992-07-05', 60000, 'Hadapsar'),


(4, 'Kiran', '1990-11-18', 70000, 'ShivajiNagar'),
(5, 'Rohit', '1999-01-09', 25000, 'Camp');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. List details of people whose name starts with 'R'
SELECT *
FROM Person
WHERE pname LIKE 'R%';

ii. List names of people whose birthday is in July
SELECT pname, birthdate
FROM Person
WHERE EXTRACT(MONTH FROM birthdate) = 7;

iii. Display details of people sorted by income
SELECT *
FROM Person
ORDER BY income;



iv. Count areas of 'urban' type
SELECT COUNT(*) AS urban_count
FROM Area
WHERE area_type = 'urban';

v. Change pincode of 'Kalyaninagar' to 411036
UPDATE Area
SET pincode = 411036
WHERE aname = 'Kalyaninagar';
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. People in 'Camp' with income < at least one person in
'Kalyaninagar'
SELECT pname
FROM Person


WHERE aname = 'Camp'
  AND income < ANY (
      SELECT income
      FROM Person
      WHERE aname = 'Kalyaninagar'
  );


2. Create a view for the person with second maximum income
CREATE VIEW SecondMaxIncome AS
SELECT *
FROM Person
WHERE income = (
    SELECT MAX(income)
    FROM Person
    WHERE income < (SELECT MAX(income) FROM Person)
);
```

SLIP 9 [Rank #2 of 25]

TIER 1 ⏱️ 10-12 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Write a menu driven program for the following operations on singly linked list.
iii. Create and display a linked list of n nodes.
iv. Delete a particular element from a linked list. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.
i) Accept & Display Function

ii) Linear Search Function ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables ( primary keys are underlined).
Person (pnumber, pname, birthdate, income),
Area( aname,area_type).
There exists a one-to-many relationship between Area and Person.
Constraint: ‘area_type’ should have value either urban or rural.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List the details of persons with income greater than 10000.
ii. Transfer all people living in ‘Pune’ to ‘Delhi’.
iii. Count number of area’s which has type ‘urban’.
iv. Give the count of people who are born in the month of February.
v. List the names of all people living in ‘rural’ area.
C. Write a query that count the total number of people staying in “urban” area with income
equal to average income.
[2 marks]
OR
C. Create a view to print the person’s details along with their area name.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Linear Search with Modular Functions: This program splits standard linear search into 3 clean, reusable functions: `accept(arr, n)` reads the input, `display(arr, n)` prints the array, and `search(arr, n, key)` scans the array and returns the index where the key was found (or -1 if missing). Examiners award top marks for modular functions because it proves you understand parameter passing in C!

🔹 **DBMS & PostgreSQL Logic:** Scenario: City Planning Registry (Person & Area). Area is parent, Person is child with Foreign Key `aname REFERENCES Area(aname)`. For queries: Query (i) lists high-income citizens with `WHERE income > 50000`. Query (ii) computes average income in urban areas using `SELECT AVG(income) FROM Person JOIN Area ... WHERE area_type = 'urban'`. Query (iii) counts persons per area using `GROUP BY aname`.

🧠 **Mental Formula & Memory Trick:** The 3 Specialist Functions: accept() fills array, display() prints array, linearSearch() does the for loop and returns index. In main(), just call them in order 1-2-3.

💡 **Winning Exam Strategy:** Avoids Singly Linked List deletion edge cases (deleting head, tail, middle). Modular array functions are trivial to code and examiner-friendly.

💻 Section I: Data Structures (13 Marks) — Linear Search with Modular Functions (Accept, Display, Search)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function 1: Populate random numbers
void accept(int arr[], int n) {
    srand(time(0));
    for (int i = 0; i < n; i++) arr[i] = rand() % 100;
}

// Function 2: Print numbers
void display(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

// Function 3: Linear search
int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) return i; // Found at index i
    }
    return -1; // Not found
}

int main() {
    int n, x;
    printf("Enter size of array: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    accept(arr, n);
    printf("Array: ");
    display(arr, n);

    printf("Enter value to search: ");
    scanf("%d", &x);

    int pos = linearSearch(arr, n, x);
    if (pos != -1)
        printf("Element %d found at position %d (index %d)\n", x, pos + 1, pos);
    else
        printf("Element %d not found in array\n", x);

    return 0;
}
```

Expected Terminal Output:

```
Enter size of array: 5
Array: 33 19 82 45 70
Enter value to search: 82
Element 82 found at position 3 (index 2)
```

🗄️ Section II: Database Management Systems (12 Marks) — Person & Area (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Area (
    aname VARCHAR(50) PRIMARY KEY,
    area_type VARCHAR(10) CHECK (area_type IN ('urban','rural'))
);



-- Person table (each person belongs to an area)
CREATE TABLE Person (
    pnumber INT PRIMARY KEY,
    pname VARCHAR(50),
    birthdate DATE,
    income NUMERIC(10,2),
    aname VARCHAR(50),
    FOREIGN KEY (aname) REFERENCES Area(aname)
);

-- Insert into Area
INSERT INTO Area VALUES
('Pune', 'urban'),
('Delhi', 'urban'),
('Nashik', 'rural'),
('Camp', 'urban');

-- Insert into Person
INSERT INTO Person VALUES
(101, 'Ravi', '1995-02-15', 12000, 'Pune'),
(102, 'Meena', '1997-07-21', 8000, 'Nashik'),
(103, 'Amit', '2000-02-05', 10000, 'Delhi'),
(104, 'Sonal', '1999-12-11', 15000, 'Pune'),
(105, 'Kiran', '2001-06-25', 12000, 'Camp');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Persons with income > 10000
SELECT *
FROM Person
WHERE income > 10000;

ii. Transfer all people from Pune → Delhi
UPDATE Person
SET aname = 'Delhi'
WHERE aname = 'Pune';

iii. Count areas of type 'urban'
SELECT COUNT(*) AS total_urban


FROM Area
WHERE area_type = 'urban';

iv. Count people born in February
SELECT COUNT(*) AS feb_birthdays
FROM Person
WHERE EXTRACT(MONTH FROM birthdate) = 2;

v. List names of people living in rural areas
SELECT pname
FROM Person
WHERE aname IN (
    SELECT aname FROM Area WHERE area_type = 'rural'
);
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Count people in urban areas with income equal to average
income
SELECT COUNT(*) AS total_people
FROM Person
WHERE income = (SELECT AVG(income) FROM Person)
  AND aname IN (SELECT aname FROM Area WHERE area_type = 'urban');


2. View: Person details with area name
CREATE VIEW PersonAreaView AS
SELECT P.pnumber, P.pname, P.birthdate, P.income, A.aname,
A.area_type
FROM Person P, Area A
WHERE P.aname = A.aname;
```

SLIP 8 [Rank #3 of 25]

TIER 1 ⏱️ 12-15 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q1) Write a menu driven program for the following operations on doubly link list.
i.
Create and Display a link list of n nodes
ii.
Insert an element in a linked list in a particular position ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following Student-Teacher database(primary keys are underlined).
Student (sno , s_name, s_class)
Teacher (tno , t_name, yrs_experience )
The relationship is as follows:
Student-Teacher: Many to Many with descriptive attribute Subject.
Constraint:s_class should be either "FY", "SY" or "TY"
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Give class-wise number of students.
ii. List all students studying in class “TY”.
iii. Count the number of students who have taken subject “Maths”
iv. Delete record of student whose sno “105”
v. Add designation column in teacher table
C. Write a query to list the names of all teachers with their subjects along with the total
number of students they are teaching.
[2 marks]
OR
C. Create a view to find the teacher who teaches maximum number of subjects.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Bubble Sort: In every pass, we compare neighbor pairs `arr[j]` and `arr[j+1]`. If the left neighbor is greater than the right neighbor, we swap them! By doing this across the whole array, the biggest number floats all the way to the end like a giant air bubble. Repeating this for n-1 passes guarantees the entire array is sorted in ascending order.

🔹 **DBMS & PostgreSQL Logic:** Scenario: University Courses. Students take courses taught by Teachers (Many-to-Many). Junction table `Student_Teacher` records `roll_no`, `tno`, and `subject`. For queries: Query (i) lists teachers residing in a given city. Query (ii) counts enrolled students. Query (iii) finds all teachers assigned to 'Mathematics'.

🧠 **Mental Formula & Memory Trick:** The Sinking Rock: Compare adjacent neighbors arr[j] and arr[j+1]. If left is bigger than right, swap them! The inner loop condition is j < n - 1 - i because after i passes, the i biggest rocks have already sunk to the back.

💡 **Winning Exam Strategy:** Avoids Doubly Linked List node creation, prev/next pointer adjustments, and menu loops. Bubble sort is 20 lines of simple nested loops.

💻 Section I: Data Structures (13 Marks) — Bubble Sort on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, temp;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0));

    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    // [MEMORY TRICK]: Adjacent neighbors compare and swap
    for (int i = 0; i < n - 1; i++) {
        // Inner loop shrinks by i each time
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap neighbors
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    printf("Sorted Array (Bubble Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 5
Random Array: 58 12 90 34 21
Sorted Array (Bubble Sort): 12 21 34 58 90
```

🗄️ Section II: Database Management Systems (12 Marks) — Student & Teacher (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Student (
    sno INT PRIMARY KEY,
    s_name VARCHAR(50),
    s_class VARCHAR(5) CHECK (s_class IN ('FY','SY','TY'))
);



-- Teacher table
CREATE TABLE Teacher (
    tno INT PRIMARY KEY,
    t_name VARCHAR(50),
    yrs_experience INT
);

-- Relationship table with descriptive attribute (Subject)
CREATE TABLE Student_Teacher (
    sno INT,
    tno INT,
    subject VARCHAR(50),
    PRIMARY KEY (sno, tno, subject),
    FOREIGN KEY (sno) REFERENCES Student(sno),
    FOREIGN KEY (tno) REFERENCES Teacher(tno)
);

-- Insert Students
INSERT INTO Student VALUES
(101, 'Ravi', 'FY'),


(102, 'Meena', 'SY'),
(103, 'Amit', 'TY'),
(104, 'Sonal', 'TY'),
(105, 'Kiran', 'SY');

-- Insert Teachers
INSERT INTO Teacher VALUES
(201, 'Prof. Patil', 10),
(202, 'Prof. Mehta', 7),
(203, 'Prof. Sharma', 12);

-- Insert Student-Teacher with subjects
INSERT INTO Student_Teacher VALUES
(101, 201, 'DBMS'),
(102, 201, 'Maths'),
(103, 202, 'Maths'),
(104, 203, 'OS'),
(105, 202, 'Maths'),
(103, 201, 'DBMS');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Class-wise number of students
SELECT s_class, COUNT(*) AS total_students
FROM Student
GROUP BY s_class;

ii. All students in TY
SELECT *
FROM Student
WHERE s_class = 'TY';

iii. Count students who have taken Maths
SELECT COUNT(DISTINCT sno) AS maths_students
FROM Student_Teacher
WHERE subject = 'Maths';

iv. Delete record of student with sno = 105


DELETE FROM Student
WHERE sno = 105;

v. Add designation column in teacher table
ALTER TABLE Teacher
ADD COLUMN designation VARCHAR(50);
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Teachers with subjects & total number of students
SELECT T.t_name, ST.subject, COUNT(DISTINCT ST.sno) AS
total_students
FROM Teacher T, Student_Teacher ST
WHERE T.tno = ST.tno
GROUP BY T.t_name, ST.subject;




2. View: Teacher who teaches maximum number of subjects
CREATE VIEW MaxSubjectsTeacher AS
SELECT tno, COUNT(DISTINCT subject) AS subject_count
FROM Student_Teacher
GROUP BY tno
ORDER BY subject_count DESC
LIMIT 1;
```

SLIP 18 [Rank #4 of 25]

TIER 1 ⏱️ 12-15 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q1) Write a program to perform the following operation on stack.
Convert an infix expression of the form (a*(b-c*d)/((a+d)/b)) into its equivalent postfix notation. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database(Primary keys are underlined)
Student (rno,name,city)
Teacher(tno,tname,phone_no,salary)
The relationship is as follows:
Student-Teacher: Many-to-Many with subject as a descriptive attribute.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List the students from “Sangamner”city.
ii. Find the maximum salary of teachers.
iii. Change the phone number of teacher ‘Dr. Mulay’ to 9834233235.
iv. List the details of the teachers in the sorted order of their name.
v. Count the number of students who have taken subject “IKS”.
C. Write a query to list the names of all students who are taught by teacher “Dr.Wani”
[2 marks]
OR
C. Create a view to list the details of students who have not taken ‘Electronics ‘subject.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Bubble Sort on Random Array: Compares adjacent elements `arr[j]` and `arr[j+1]` across nested loops. Swaps when left > right. After n-1 passes, all numbers are in perfect ascending order. Easiest algorithm to recall and write from muscle memory during the exam.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Student & Teacher (Many-to-Many). Same academic structure as Slip 8. For queries: Query (i) lists teachers residing in Pune. Query (ii) counts total registered students. Query (iii) updates teacher contact details.

🧠 **Mental Formula & Memory Trick:** Sinking Neighbors: Identical to Slip 8! Two nested loops: for i=0 to n-1, for j=0 to n-1-i. If arr[j] > arr[j+1], swap! Avoids Infix-to-Postfix conversion.

💡 **Winning Exam Strategy:** Option A avoids tedious Infix-to-Postfix conversion (operator precedence, associativity, parenthesis balancing). Bubble Sort is 20 lines of array code.

💻 Section I: Data Structures (13 Marks) — Bubble Sort on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, temp;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0));

    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    printf("Sorted Array (Bubble Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 5
Random Array: 88 12 43 5 29
Sorted Array (Bubble Sort): 5 12 29 43 88
```

🗄️ Section II: Database Management Systems (12 Marks) — Student & Teacher (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Student (
    rno INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50)
);

-- Create Teacher table
CREATE TABLE Teacher (
    tno INT PRIMARY KEY,
    tname VARCHAR(50),
    phone_no VARCHAR(15),
    salary INT
);

-- Create Relationship table (M-M with Subject)
CREATE TABLE StudentTeacher (
    rno INT,


    tno INT,
    subject VARCHAR(50),
    PRIMARY KEY (rno, tno, subject),
    FOREIGN KEY (rno) REFERENCES Student(rno),
    FOREIGN KEY (tno) REFERENCES Teacher(tno)
);

-- Insert Students
INSERT INTO Student VALUES (1, 'Amit', 'Sangamner');
INSERT INTO Student VALUES (2, 'Sneha', 'Pune');
INSERT INTO Student VALUES (3, 'Ravi', 'Nashik');
INSERT INTO Student VALUES (4, 'Meera', 'Sangamner');

-- Insert Teachers
INSERT INTO Teacher VALUES (101, 'Dr. Mulay', '9876543210', 50000);
INSERT INTO Teacher VALUES (102, 'Dr. Wani', '9123456789', 60000);
INSERT INTO Teacher VALUES (103, 'Dr. Patil', '9988776655', 45000);

-- Insert Relationship (Student-Teacher-Subject)
INSERT INTO StudentTeacher VALUES (1, 101, 'Maths');
INSERT INTO StudentTeacher VALUES (2, 102, 'IKS');
INSERT INTO StudentTeacher VALUES (3, 101, 'Electronics');
INSERT INTO StudentTeacher VALUES (4, 102, 'IKS');
INSERT INTO StudentTeacher VALUES (2, 103, 'Physics');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. List the students from "Sangamner" city
SELECT * FROM Student
WHERE city = 'Sangamner';


ii. Find the maximum salary of teachers
SELECT MAX(salary) AS max_salary
FROM Teacher;


iii. Change the phone number of teacher 'Dr. Mulay' to 9834233235
UPDATE Teacher
SET phone_no = '9834233235'


WHERE tname = 'Dr. Mulay';


iv. List details of teachers in sorted order of their name
SELECT * FROM Teacher
ORDER BY tname;


v. Count the number of students who have taken subject "IKS"
SELECT COUNT(DISTINCT rno) AS student_count
FROM StudentTeacher
WHERE subject = 'IKS';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: List the names of all students taught by “Dr. Wani”
SELECT s.name
FROM Student s
JOIN StudentTeacher st ON s.rno = st.rno
JOIN Teacher t ON st.tno = t.tno
WHERE t.tname = 'Dr. Wani';


Option 2: View for students who have not taken ‘Electronics’
CREATE VIEW NoElectronicsStudents AS
SELECT s.rno, s.name, s.city
FROM Student s
WHERE s.rno NOT IN (
    SELECT rno FROM StudentTeacher
    WHERE subject = 'Electronics'
);
```

SLIP 6 [Rank #5 of 25]

TIER 1 ⏱️ 12-15 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Write a menu driven program for the following operations on singly link list.
i. Create and display a linked list of n nodes.
ii. Search an element in a linked list. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database( primary keys are underlined).
Student (rno,name,city)
Teacher(tno,tname,phone_no,salary)
The relationship is as follows: Student-Teacher: many-to-many with subject as a
descriptive attribute.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List the names of students from ‘Ahilyanagar’ city.
ii. Display the count of teachers whose salary is between 40000 to 60000.
iii. Change the phone number of ‘Prof. Patil’ to ‘9822131226’
iv. List the details of the teachers in the sorted order of their name.
v. List the names of the teachers who have salary less than 50000.
C. Write a query to list the names of all teachers who do not teach any subject that is
taught by ‘Mr. Patil’.
[2 marks]
OR
C. Create a view to list the details of students who have taken ‘DBMS’ subject.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Selection Sort sorts by picking the smallest number and putting it at the front. Imagine looking through an unsorted line of students: scan from position 0 to the end, find the shortest student, and swap them with the student at position 0. Then scan from position 1 to the end, find the next shortest, and swap with position 1. Outer loop `i` advances the boundary, inner loop `j` searches for the minimum index `min_idx`, and we swap `arr[i]` with `arr[min_idx]`.

🔹 **DBMS & PostgreSQL Logic:** Scenario: School Academic System. Multiple Students are assigned to a Teacher (Many-to-One). Teacher is parent (`tno PRIMARY KEY`), Student has `tno REFERENCES Teacher(tno)`. For queries: Query (i) lists students assigned to a specific teacher using `WHERE tno = ...`. Query (ii) counts students per teacher. Query (iii) updates a teacher's qualification using `UPDATE`.

🧠 **Mental Formula & Memory Trick:** The Talent Scout: In each round i, assume slot i is the smallest: min_idx = i. Look at everyone ahead (j = i+1 to n). If arr[j] is smaller, update min_idx = j. At the end of the round, make ONE swap between arr[i] and arr[min_idx].

💡 **Winning Exam Strategy:** Selection Sort on array is only 25 lines of code. Massive time-saver compared to implementing a full menu-driven singly linked list with insertion.

💻 Section I: Data Structures (13 Marks) — Selection Sort on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, min_idx, temp;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0));

    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    // [MEMORY TRICK]: Outer loop picks slot i, Inner loop finds minimum index
    for (int i = 0; i < n - 1; i++) {
        min_idx = i; // Assume current index is smallest
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j; // Found a smaller candidate!
        }
        // One single swap per pass
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }

    printf("Sorted Array (Selection Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 5
Random Array: 64 25 12 22 11
Sorted Array (Selection Sort): 11 12 22 25 64
```

🗄️ Section II: Database Management Systems (12 Marks) — Student & Teacher (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Student (
    rno INT PRIMARY KEY,


    name VARCHAR(50),
    city VARCHAR(50)
);

-- Teacher table
CREATE TABLE Teacher (
    tno INT PRIMARY KEY,
    tname VARCHAR(50),
    phone_no VARCHAR(15),
    salary NUMERIC(10,2)
);

-- Relationship table with subject
CREATE TABLE Student_Teacher (
    rno INT,
    tno INT,
    subject VARCHAR(50),
    PRIMARY KEY (rno, tno, subject),
    FOREIGN KEY (rno) REFERENCES Student(rno),
    FOREIGN KEY (tno) REFERENCES Teacher(tno)


);

-- Insert Students
INSERT INTO Student VALUES
(1, 'Ravi', 'Pune'),
(2, 'Meena', 'Ahilyanagar'),
(3, 'Arjun', 'Mumbai'),
(4, 'Sita', 'Ahilyanagar');

-- Insert Teachers
INSERT INTO Teacher VALUES
(101, 'Prof. Patil', '9876543210', 45000),
(102, 'Mr. Kumar', '9898989898', 55000),
(103, 'Dr. Mehta', '9123456789', 65000),
(104, 'Mr. Joshi', '9234567890', 40000);

-- Insert Student-Teacher relations with subjects
INSERT INTO Student_Teacher VALUES
(1, 101, 'DBMS'),
(2, 101, 'DBMS'),


(3, 102, 'Maths'),
(4, 104, 'English'),
(2, 103, 'Science'),
(1, 104, 'English');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. List the names of students from 'Ahilyanagar'
SELECT name
FROM Student
WHERE city = 'Ahilyanagar';

ii. Count of teachers whose salary is between 40000 to 60000
SELECT COUNT(*) AS teacher_count
FROM Teacher
WHERE salary BETWEEN 40000 AND 60000;

iii. Change phone number of 'Prof. Patil'


UPDATE Teacher
SET phone_no = '9822131226'
WHERE tname = 'Prof. Patil';

iv. List teachers sorted by name
SELECT *
FROM Teacher
ORDER BY tname;

v. Names of teachers with salary < 50000
SELECT tname
FROM Teacher
WHERE salary < 50000;
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Teachers who do not teach any subject taught by 'Mr. Patil'


SELECT tname
FROM Teacher
WHERE tno NOT IN (
    SELECT tno
    FROM Student_Teacher
    WHERE subject IN (
        SELECT subject
        FROM Student_Teacher
        WHERE tno = (
            SELECT tno FROM Teacher WHERE tname = 'Mr. Patil'
        )
    )
);


2. Create a view: students who have taken 'DBMS' subject
CREATE VIEW DBMS_Students AS
SELECT S.rno, S.name, S.city
FROM Student S


WHERE S.rno IN (
    SELECT rno
    FROM Student_Teacher
    WHERE subject = 'DBMS'
);
```

SLIP 12 [Rank #6 of 25]

TIER 1 ⏱️ 12-15 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Perform the following operations using dynamic implementation of stack
i. Accept n integers from user and push it into the stack.
ii. Reverses a string of characters using stack and check weather given string is palindrome or not. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables (primary keys are underlined).
Sales_order(sorderno, s_order_date, order_amt)
Client(clientno, name, address)
There exists a one-to-many relationship between Client and Sales_order
Constraints: order_amt should be greater than zero.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Update the client address of all clients from “Nagpur” to“Kolhapur”.
ii. Delete all sales order whose client number is 30;
iii. Display all sale records having order date before “23/02/2024”.
iv. Display sales order having maximum order amount.
v. Add column order_status to the Sales_order table.
C. Write a query to Display client details having maximum sales orders.
[2 marks]
OR
C. Create a view to list names of clients having more than 2 sales orders.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Selection Sort on Random Integers: Instead of implementing dynamic stacks and palindrome checking with linked lists, we choose the Selection Sort option! We fill an array with random numbers, find the minimum element in each pass, and swap it to the front. It takes 25 lines of clean C code and is finished in 10 minutes flat.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Order Management. Clients place Sales Orders (Many-to-One). Sales_order table references `Client(clientno)` and has a constraint `CHECK (order_amt > 0)`. For queries: Query (i) updates client city with `UPDATE Client SET address = 'Kolhapur' WHERE address = 'Nagpur'`. Query (ii) deletes orders for client 30 with `DELETE FROM Sales_order WHERE clientno = 30`. Query (iii) finds the biggest order with `SELECT MAX(order_amt) FROM Sales_order`.

🧠 **Mental Formula & Memory Trick:** Find the Minimum & Swap: Identical to Slip 6! min_idx = i. Inner loop j finds smaller element index. Single swap at end of round i.

💡 **Winning Exam Strategy:** Option B avoids pointer-based dynamic stack and palindrome logic. Selection sort on array runs in 25 lines of direct index comparisons.

💻 Section I: Data Structures (13 Marks) — Selection Sort on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, min_idx, temp;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0));

    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    for (int i = 0; i < n - 1; i++) {
        min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) min_idx = j;
        }
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }

    printf("Sorted Array (Selection Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 5
Random Array: 45 12 89 23 7
Sorted Array (Selection Sort): 7 12 23 45 89
```

🗄️ Section II: Database Management Systems (12 Marks) — Sales_order & Client (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Client (
    clientno INT PRIMARY KEY,
    name VARCHAR(50),
    address VARCHAR(100)
);

-- Sales_order table
CREATE TABLE Sales_order (
    sorderno INT PRIMARY KEY,
    s_order_date DATE,
    order_amt NUMERIC(12,2) CHECK (order_amt > 0),
    clientno INT,
    FOREIGN KEY (clientno) REFERENCES Client(clientno)
);

-- Insert Clients
INSERT INTO Client VALUES
(10, 'Ravi', 'Nagpur'),
(20, 'Meena', 'Pune'),
(30, 'Amit', 'Mumbai'),
(40, 'Sonal', 'Nagpur');



-- Insert Sales Orders
INSERT INTO Sales_order VALUES
(101, '2024-02-20', 12000, 10),
(102, '2024-02-25', 15000, 20),
(103, '2024-01-15', 8000, 30),
(104, '2024-02-22', 25000, 10),
(105, '2024-03-01', 20000, 40);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```sql
B. Queries (any 3)
i. Update Nagpur → Kolhapur
UPDATE Client
SET address = 'Kolhapur'
WHERE address = 'Nagpur';

ii. Delete all sales orders for clientno = 30
DELETE FROM Sales_order
WHERE clientno = 30;

iii. Sales records before 23/02/2024
SELECT *
FROM Sales_order
WHERE s_order_date < '2024-02-23';

iv. Sales order with maximum amount
SELECT *
FROM Sales_order
WHERE order_amt = (SELECT MAX(order_amt) FROM Sales_order);

v. Add column order_status
ALTER TABLE Sales_order
ADD COLUMN order_status VARCHAR(20);
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Client with maximum sales orders
SELECT C.clientno, C.name, C.address, COUNT(S.sorderno) AS
total_orders
FROM Client C, Sales_order S
WHERE C.clientno = S.clientno
GROUP BY C.clientno, C.name, C.address
ORDER BY total_orders DESC
LIMIT 1;


2. View: Clients having more than 2 sales orders
CREATE VIEW ClientsWithMoreThan2Orders AS
SELECT C.clientno, C.name, COUNT(S.sorderno) AS total_orders
FROM Client C, Sales_order S
WHERE C.clientno = S.clientno
GROUP BY C.clientno, C.name
HAVING COUNT(S.sorderno) > 2;
```

SLIP 1 [Rank #7 of 25]

TIER 1 ⏱️ 12-15 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Write a C program to create random n elements and sort it in ascending order by using insertion sort. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Write a C program to perform the following operations using static implementation of stack
i. Accept n integers from user and push it into the stack.
ii. Pop the element from the stack and display it (Use isEmpty()) 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database( primary keys are underlined).
Room (roomno,room_name,room_type,charges)
Guest(Guestcode, Gname,city)
The relationship is as follows: Room-Guest: one-to-one. room_type can have values as either
'AC'or 'NonAC'.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List the details of the rooms having charges between 5000 and 10000.
ii. List the names of the guests in the sorted order by city name.
iii. List the minimum charges of a room.
iv. Increase the charges of all AC rooms by 15%.
v. List the names of all the NONAC rooms whose charges are more
than 10000.
C. Write a query to list the name of the guest to whom the room with highest charges is allotted.
[2 marks]
OR
C. Create a view to list the names of all the NonAC rooms which have
charges greater than at least one of the 'AC' room.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** A Stack is a 'Last In, First Out' (LIFO) collection — imagine a vertical stack of cafeteria plates. You can only place a plate on top (PUSH) and take a plate from the top (POP). We use a static array `stack[MAX]` and an integer variable `top` which starts at -1 (empty floor). When pushing, we do `top++` first, then put the number in `stack[top]`. When popping, we take the number out of `stack[top]`, then do `top--`. `isEmpty()` simply checks if `top == -1`.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Hotel Room Reservation. We have Rooms and Guests. The relationship is One-to-One (each guest gets at most 1 room). That's why in the Guest table, `roomno INT UNIQUE REFERENCES Room(roomno)` is marked UNIQUE! For queries: Query (i) finds mid-price rooms using `BETWEEN 5000 AND 10000`. Query (ii) sorts guests alphabetically by city using `ORDER BY city`. Query (iii) finds the minimum room rate using `SELECT MIN(charges) FROM Room`.

🧠 **Mental Formula & Memory Trick:** The Cafeteria Plate Rule: 'top' starts at -1 (empty floor). To push a plate, increment FIRST then place: s[++top] = x. To pop, take plate then decrement: s[top--]. Stack empty check is just top == -1.

💡 **Winning Exam Strategy:** Option B requires only a basic array and a top pointer. Writing static push/pop takes under 25 lines and completely avoids pointers, malloc, and recursion.

💻 Section I: Data Structures (13 Marks) — Static Stack (Push & Pop with isEmpty)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

// [MEMORY TRICK]: Top starts at -1 (empty floor)
int stack[MAX];
int top = -1;

// [TRICK]: Empty when top is still on the floor (-1)
int isEmpty() {
    return top == -1;
}

// [TRICK]: Move UP first (++top), then put value
void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow!\n");
        return;
    }
    stack[++top] = value;
}

// [TRICK]: Take value, then move DOWN (top--)
int pop() {
    if (isEmpty()) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--];
}

int main() {
    int n, val;
    printf("Enter number of integers to push: ");
    if (scanf("%d", &n) != 1) return 0;

    // Step 1: Accept n elements and push into stack
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push(val);
    }

    // Step 2: Pop and display using isEmpty() (reverses order inherently!)
    printf("\nPopping elements from stack:\n");
    while (!isEmpty()) {
        printf("%d ", pop());
    }
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of integers to push: 4
Enter 4 integers: 10 25 38 50

Popping elements from stack:
50 38 25 10
```

🗄️ Section II: Database Management Systems (12 Marks) — Room & Guest (One-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Room (
    roomno INT PRIMARY KEY,
    room_name VARCHAR(50),
    room_type VARCHAR(10) CHECK (room_type IN ('AC', 'NonAC')),
    charges NUMERIC(10,2)
);

-- Create Guest table
CREATE TABLE Guest (
    Guestcode INT PRIMARY KEY,
    Gname VARCHAR(50),
    city VARCHAR(50),
    roomno INT UNIQUE,  -- one-to-one relationship
    FOREIGN KEY (roomno) REFERENCES Room(roomno)
);



-- Insert sample records into Room
INSERT INTO Room VALUES
(1, 'Deluxe1', 'AC', 8000),
(2, 'Deluxe2', 'NonAC', 6000),
(3, 'Suite1', 'AC', 12000),
(4, 'Suite2', 'NonAC', 15000);

-- Insert sample records into Guest
INSERT INTO Guest VALUES
(101, 'Ravi', 'Pune', 1),
(102, 'Meena', 'Mumbai', 2),
(103, 'Arjun', 'Delhi', 3),
(104, 'Kiran', 'Bangalore', 4);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. List details of rooms with charges between 5000 and 10000
SELECT *
FROM Room
WHERE charges BETWEEN 5000 AND 10000;

ii. List names of guests in sorted order by city
SELECT Gname, city
FROM Guest
ORDER BY city;

iii. List the minimum charges of a room
SELECT MIN(charges) AS min_charges
FROM Room;

iv. Increase charges of all AC rooms by 15%


UPDATE Room
SET charges = charges * 1.15
WHERE room_type = 'AC';

v. List names of all NonAC rooms with charges > 10000
SELECT room_name
FROM Room
WHERE room_type = 'NonAC' AND charges > 10000;
```

Part C: Advanced Query / View [2 Marks]

```
C. Query for highest charges room's guest
SELECT Gname
FROM Guest
WHERE roomno = (
    SELECT roomno
    FROM Room
    WHERE charges = (SELECT MAX(charges) FROM Room)
);



Alternative C. Create a View
CREATE VIEW NonAC_View AS
SELECT room_name, charges
FROM Room
WHERE room_type = 'NonAC'
  AND charges > ANY (SELECT charges FROM Room WHERE room_type = 'AC');
```

SLIP 4 [Rank #8 of 25]

TIER 1 ⏱️ 12-15 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Read the data from file 'student.txt' containing names of student and their class (FY, SY, TY). Accept a student name from user and use linear search algorithm to check whether the name is present in the file and output the Class of student otherwise output "Student not in the list". ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.
👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q.2) Consider the following database (primary keys are underlined).
Policy (pno, pname,premium_amt,policy_type)
Customer (cno,cname.city, agent_name)
The relationship is as follows: Policy-Customer: many-to-one. The 'policy_type' can have values as
'Yearly', 'Half-yearly' or 'Monthly'.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostgreSQL
(any 3)
[6 marks]
i. List the details of all customers who live in city.
ii. Display the average premium amount.
iii. Count the number of customers who have taken 'Jeevan Anand' policy.
iv. Increase the premium amount for 'Monthly 'policies by 10%.
v. Display the policy_type wise count of policies.
C. Write a query to list the policy_type with highest average premium amount.
[2 marks]
OR
C. Create a view to list the name of the customer who has taken highest number of policies.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Reversing a string with a Stack: Why does a stack reverse text? Because whatever you push first sinks to the bottom, and whatever you push last sits right at the top! If you push the letters 'C', 'A', 'T' onto the stack, 'T' is on top. Popping them gives 'T', 'A', 'C' ('TAC'). In C, we push all characters using a loop, then pop them off into a new string until `top == -1`. It takes only 25 lines and is 100% immune to runtime errors!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Life Insurance Agency. Multiple insurance Policies belong to a Customer (Many-to-One). Customer is parent (`cno PRIMARY KEY`), and Policy has `cno REFERENCES Customer(cno)`. For queries: Query (i) filters high-premium policies with `WHERE premium_amt > 10000`. Query (ii) counts policies per customer. Query (iii) updates policy terms using `UPDATE`.

🧠 **Mental Formula & Memory Trick:** The Echo Chamber: Feed characters into a character stack: stack[++top] = str[i]. Pop them out one by one: stack[top--]. Because stack is Last-In-First-Out, popping inherently reverses the entire word!

💡 **Winning Exam Strategy:** Avoids file handling (`fopen`, `fscanf`, string tokens) of student.txt. Character stack reversal takes just 25 lines and works instantly.

💻 Section I: Data Structures (13 Marks) — Reverse String using Static Stack

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <string.h>
#define MAX 100

// [MEMORY TRICK]: Stack of characters
char stack[MAX];
int top = -1;

void push(char ch) {
    if (top < MAX - 1)
        stack[++top] = ch; // Move up, store char
}

char pop() {
    if (top >= 0)
        return stack[top--]; // Return char, move down
    return '\0';
}

int main() {
    char str[MAX];
    printf("Enter string to reverse: ");
    if (scanf(" %[^\n]", str) != 1) return 0;

    int len = strlen(str);

    // [STEP 1]: Push each character onto stack
    for (int i = 0; i < len; i++) {
        push(str[i]);
    }

    // [STEP 2]: Pop characters back (comes out reversed!)
    printf("Reversed string: ");
    while (top != -1) {
        printf("%c", pop());
    }
    printf("\n");

    return 0;
}
```

Expected Terminal Output:

```
Enter string to reverse: Computer Science
Reversed string: ecneicS retupmoC
```

🗄️ Section II: Database Management Systems (12 Marks) — Policy & Customer (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Policy (
    pno INT PRIMARY KEY,
    pname VARCHAR(50),
    premium_amt NUMERIC(10,2),
    policy_type VARCHAR(20) CHECK (policy_type IN ('Yearly', 'Half-
yearly', 'Monthly'))
);



-- Create Customer table (many-to-one: each customer is linked to one
policy)
CREATE TABLE Customer (
    cno INT PRIMARY KEY,
    cname VARCHAR(50),
    city VARCHAR(50),
    agent_name VARCHAR(50),
    pno INT,
    FOREIGN KEY (pno) REFERENCES Policy(pno)
);

-- Insert sample records into Policy
INSERT INTO Policy VALUES
(1, 'Jeevan Anand', 25000, 'Yearly'),
(2, 'Health Plus', 12000, 'Half-yearly'),
(3, 'Child Secure', 2000, 'Monthly'),
(4, 'Life Secure', 5000, 'Monthly'),
(5, 'Retirement Plan', 30000, 'Yearly');



-- Insert sample records into Customer
INSERT INTO Customer VALUES
(101, 'Ravi', 'Pune', 'Ajay', 1),
(102, 'Meena', 'Mumbai', 'Vikas', 2),
(103, 'Kiran', 'Delhi', 'Sonia', 3),
(104, 'Arjun', 'Pune', 'Ajay', 4),
(105, 'Sita', 'Nashik', 'Vikas', 1),
(106, 'Rohit', 'Pune', 'Sonia', 5);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. List details of all customers who live in Pune
SELECT *
FROM Customer
WHERE city = 'Pune';

ii. Display the average premium amount
SELECT AVG(premium_amt) AS avg_premium


FROM Policy;

iii. Count customers who have taken 'Jeevan Anand' policy
SELECT COUNT(*) AS customer_count
FROM Customer
WHERE pno = (
    SELECT pno
    FROM Policy
    WHERE pname = 'Jeevan Anand'
);

iv. Increase premium amount for 'Monthly' policies by 10%
UPDATE Policy
SET premium_amt = premium_amt * 1.10
WHERE policy_type = 'Monthly';

v. Policy_type wise count of policies
SELECT policy_type, COUNT(*) AS total_policies
FROM Policy


GROUP BY policy_type;
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Policy_type with highest average premium amount
SELECT policy_type
FROM Policy
GROUP BY policy_type
ORDER BY AVG(premium_amt) DESC
LIMIT 1;


2. View: customer with highest number of policies
CREATE VIEW MaxPoliciesCustomer AS
SELECT cname, COUNT(*) AS total_policies
FROM Customer
GROUP BY cname
ORDER BY total_policies DESC


LIMIT 1;
```

SLIP 11 [Rank #9 of 25]

TIER 1 ⏱️ 12-15 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.
👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Read the data from the 'employee.txt' file and sort on age using Counting sort and write the sorted data to another file 'sortedemponage.txt'. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables (primary keys are underlined).
Emp (eno, ename, salary)
Project (pno, pname, budget)
Relationships:
Emp- Project : Many to Many with descriptive attribute total_hours.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Find the maximum budget.
ii. Increase the salary of all employees by 10%.
iii. Count the number of projects having duration greater than 100 hrs.
iv. List all employees whose name ends with "a".
v. Add column contact_number to employee table.
C. Write a query to list the names of employees who are not working on any project.
[2 marks]
OR
C. Create a view to list the project details having maximum number of hours.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Reverse String using Array Stack: We read a word from the user (e.g. 'HELLO'). We iterate through each character and push it into an array-based character stack. Then we pop them out one by one. Because the last letter pushed ('O') is on top, it pops first, followed by 'L', 'L', 'E', 'H' ('OLLEH'). This directly fulfills the question without messing with file pointers.

🔹 **DBMS & PostgreSQL Logic:** Scenario: IT Project Management. Employees collaborate across multiple Projects (Many-to-Many). Bridge table `Emp_Project` has `empno`, `projno`, and `role`. For queries: Query (i) lists employees allocated to a specific project. Query (ii) counts active projects. Query (iii) updates project completion dates with `UPDATE`.

🧠 **Mental Formula & Memory Trick:** Mirror Reversal: Same character stack trick as Slip 4. Push every letter: stack[++top] = c. Pop while top != -1. Output is naturally backwards.

💡 **Winning Exam Strategy:** Avoids file handling with employee.txt and Counting Sort. Push and pop characters using a stack takes 25 lines and is 100% bug-free.

💻 Section I: Data Structures (13 Marks) — Reverse String using Static Stack

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <string.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top < MAX - 1) stack[++top] = c;
}

char pop() {
    if (top >= 0) return stack[top--];
    return '\0';
}

int main() {
    char str[MAX];
    printf("Enter string to reverse: ");
    if (scanf(" %[^\n]", str) != 1) return 0;

    int len = strlen(str);
    for (int i = 0; i < len; i++) push(str[i]);

    printf("Reversed string: ");
    while (top != -1) printf("%c", pop());
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter string to reverse: DataStructures
Reversed string: serutcurtSataD
```

🗄️ Section II: Database Management Systems (12 Marks) — Emp & Project (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Emp (
    eno INT PRIMARY KEY,


    ename VARCHAR(50),
    salary NUMERIC(10,2)
);

-- Project table
CREATE TABLE Project (
    pno INT PRIMARY KEY,
    pname VARCHAR(50),
    budget NUMERIC(12,2)
);

-- Relationship table with descriptive attribute (total_hours)
CREATE TABLE Emp_Project (
    eno INT,
    pno INT,
    total_hours INT,
    PRIMARY KEY (eno, pno),
    FOREIGN KEY (eno) REFERENCES Emp(eno),
    FOREIGN KEY (pno) REFERENCES Project(pno)
);

-- Insert Employees
INSERT INTO Emp VALUES
(1, 'Ravi', 25000),
(2, 'Meena', 30000),
(3, 'Sonia', 28000),
(4, 'Amit', 35000);

-- Insert Projects
INSERT INTO Project VALUES
(101, 'DBMS', 500000),
(102, 'AI', 800000),
(103, 'Networking', 400000);

-- Insert Emp_Project (who works where & hours)
INSERT INTO Emp_Project VALUES
(1, 101, 120),
(2, 101, 80),
(2, 102, 150),
(3, 103, 90);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Find the maximum budget


SELECT MAX(budget) AS max_budget
FROM Project;

ii. Increase salary of all employees by 10%
UPDATE Emp
SET salary = salary * 1.10;

iii. Count projects with duration > 100 hrs
SELECT COUNT(DISTINCT pno) AS project_count
FROM Emp_Project
WHERE total_hours > 100;

iv. Employees whose name ends with 'a'
SELECT *
FROM Emp
WHERE ename LIKE '%a';

v. Add column contact_number
ALTER TABLE Emp
ADD COLUMN contact_number VARCHAR(15);
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Employees not working on any project
SELECT ename
FROM Emp
WHERE eno NOT IN (SELECT eno FROM Emp_Project);


2. View: Project with maximum hours
CREATE VIEW MaxHoursProject AS
SELECT P.pno, P.pname, P.budget, SUM(EP.total_hours) AS total_hours
FROM Project P, Emp_Project EP
WHERE P.pno = EP.pno
GROUP BY P.pno, P.pname, P.budget


ORDER BY total_hours DESC
LIMIT 1;
```

TIER 2: EASY & STANDARD

Standard algorithmic and static queue/stack slips. Smooth and predictable.

SLIP 7 [Rank #10 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Read the data from "Student.txt (Stud_name, Age, Percentage)" file and sort on Age using insertion sort. (Create a separate sorted file). ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database( primary keys are underlined).
Item (itemno, name, quantity)
Supplier (sno, name, city)
The relationship is as follows: Item-Supplier: many-to-many with rate as a descriptive attribute.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Change the quantity of item 'Mouse' to 80.
ii. List the details of the suppliers whose name begins with the alphabet 'M'.
iii. Display the total count of items.
iv. List the names of suppliers who do not live in Pune city.
v. List the names of items with quantity less than 10.
C. Write a query to display the name of supplier who supplies Item with minimum rate.
[2 marks]
OR
C. Create a view to display supplier-wise list of items.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Binary Search on a pre-sorted array: Imagine searching for a word in a physical dictionary. You don't read page 1, 2, 3! You open right in the MIDDLE (`mid = (low + high) / 2`). If your word is smaller than the middle word, you discard the entire right half (`high = mid - 1`). If bigger, discard the entire left half (`low = mid + 1`). Because the array is already sorted, you cut the problem in half on every single step!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Warehouse Supply Chain. Suppliers supply multiple Items, and an Item is supplied by multiple Suppliers (Many-to-Many). Connected via bridge table `Item_Supplier` containing `itemno`, `sno`, and `rate`. For queries: Query (i) updates item stock using `UPDATE Item SET quantity = 80 WHERE name = 'Mouse'`. Query (ii) searches suppliers beginning with 'M' (`WHERE name LIKE 'M%'`). Query (iii) counts total inventory items using `SELECT COUNT(*) FROM Item`.

🧠 **Mental Formula & Memory Trick:** The Phonebook Halver: While low <= high, inspect mid = (low + high)/2. Target matched? Return mid. Target is bigger? Ignore left half (low = mid + 1). Target is smaller? Ignore right half (high = mid - 1).

💡 **Winning Exam Strategy:** Avoids file handling (`Student.txt`, structs, sorting). Binary search is classic, highly predictable, and needs only low/mid/high integer variables.

💻 Section I: Data Structures (13 Marks) — Binary Search on Sorted Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>

// [MEMORY TRICK]: Cut search space in half every step
int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2; // Find midpoint

        if (arr[mid] == x)
            return mid;             // Target found!
        else if (arr[mid] < x)
            low = mid + 1;          // Target is on the right
        else
            high = mid - 1;         // Target is on the left
    }
    return -1; // Not found
}

int main() {
    int n, x;
    printf("Enter number of sorted elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    printf("Enter %d sorted elements:\n", n);
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    printf("Enter element to search: ");
    scanf("%d", &x);

    int pos = binarySearch(arr, n, x);
    if (pos != -1)
        printf("Element %d found at position %d (index %d)\n", x, pos + 1, pos);
    else
        printf("Element %d is NOT present in the sorted array\n", x);

    return 0;
}
```

Expected Terminal Output:

```
Enter number of sorted elements: 5
Enter 5 sorted elements: 10 20 30 40 50
Enter element to search: 30
Element 30 found at position 3 (index 2)
```

🗄️ Section II: Database Management Systems (12 Marks) — Item & Supplier (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Item (
    itemno INT PRIMARY KEY,
    name VARCHAR(50),
    quantity INT
);

-- Supplier table
CREATE TABLE Supplier (
    sno INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50)
);



-- Relationship table with descriptive attribute (rate)
CREATE TABLE Item_Supplier (
    itemno INT,
    sno INT,
    rate NUMERIC(10,2),
    PRIMARY KEY (itemno, sno),
    FOREIGN KEY (itemno) REFERENCES Item(itemno),
    FOREIGN KEY (sno) REFERENCES Supplier(sno)
);

-- Insert Items
INSERT INTO Item VALUES
(1, 'Keyboard', 50),
(2, 'Mouse', 40),
(3, 'Monitor', 15),
(4, 'USB Cable', 5);

-- Insert Suppliers
INSERT INTO Supplier VALUES
(101, 'Mahesh Traders', 'Pune'),


(102, 'Meera Electronics', 'Mumbai'),
(103, 'Raj Suppliers', 'Nashik'),
(104, 'Mohan Distributors', 'Nagpur');

-- Insert Item-Supplier relationships with rate
INSERT INTO Item_Supplier VALUES
(1, 101, 500),
(2, 101, 250),
(3, 102, 7000),
(4, 103, 100),
(2, 104, 230),
(3, 104, 6800);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Change quantity of item 'Mouse' to 80
UPDATE Item
SET quantity = 80
WHERE name = 'Mouse';



ii. Suppliers whose name begins with 'M'
SELECT *
FROM Supplier
WHERE name LIKE 'M%';

iii. Total count of items
SELECT COUNT(*) AS total_items
FROM Item;

iv. Suppliers not living in Pune
SELECT name
FROM Supplier
WHERE city <> 'Pune';

v. Items with quantity < 10
SELECT name


FROM Item
WHERE quantity < 10;
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Supplier who supplies Item with minimum rate
SELECT S.name
FROM Supplier S
WHERE S.sno = (
    SELECT sno
    FROM Item_Supplier
    WHERE rate = (SELECT MIN(rate) FROM Item_Supplier)
);


2. View: supplier-wise list of items
CREATE VIEW Supplier_Item_List AS
SELECT S.name AS supplier_name, I.name AS item_name, IS1.rate


FROM Supplier S, Item I, Item_Supplier IS1
WHERE S.sno = IS1.sno AND I.itemno = IS1.itemno;
```

SLIP 13 [Rank #11 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Perform the following operations using dynamic implementation of Queue
i. Insert n elements in a queue.
ii. Delete the element from the queue and display it. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables (primary keys are underlined).
Bus ( Busno , capacity ,depot_name)
Route (Routeno ,source ,destination ,no_of_stations )
Relationship: Bus-Route : Many to one
Constraint: Bus capacity should not be null.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List all buses at depot “Swargate”.
ii. Delete Bus whose Bus number is “204”.
iii. List all buses on route number 41.
iv. List the route details having number of stations greater than 10.
v. List all routes starting from “Pune station”.
C. Write a query to delete all the buses on routes in which total stations are less than 3.
[2 marks]
OR
C. Create a view to list all buses and route details from “Pimpri”to “Hadapsar”.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Binary Search on Sorted Array: Accepts n numbers in sorted order from the user. We calculate `mid = (low + high) / 2`. If `arr[mid] == x`, we report success. If `x < arr[mid]`, search left (`high = mid - 1`); if `x > arr[mid]`, search right (`low = mid + 1`). Runs in O(log n) time and is super easy to reproduce under exam stress.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Public Bus Transport. Multiple Buses operate along assigned transit Routes (Many-to-One). Bus table references `Route(route_no)`. For queries: Query (i) lists buses on a specific route. Query (ii) counts buses with passenger capacity > 40. Query (iii) updates depot location with `UPDATE`.

🧠 **Mental Formula & Memory Trick:** The Halving Loop: Identical to Slip 7! low = 0, high = n - 1, mid = (low + high)/2. If arr[mid] == x return mid. If arr[mid] < x, low = mid + 1. Else high = mid - 1.

💡 **Winning Exam Strategy:** Option A avoids dynamic linked queue memory allocations and pointer adjustments. Standard binary search loop is reliable and quick.

💻 Section I: Data Structures (13 Marks) — Binary Search on Sorted Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>

int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == x) return mid;
        else if (arr[mid] < x) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int n, x;
    printf("Enter number of sorted elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    printf("Enter %d sorted elements:\n", n);
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    printf("Enter value to search: ");
    scanf("%d", &x);

    int pos = binarySearch(arr, n, x);
    if (pos != -1)
        printf("Element %d found at position %d (index %d)\n", x, pos + 1, pos);
    else
        printf("Element %d not found in array\n", x);

    return 0;
}
```

Expected Terminal Output:

```
Enter number of sorted elements: 4
Enter 4 sorted elements: 5 15 25 35
Enter value to search: 25
Element 25 found at position 3 (index 2)
```

🗄️ Section II: Database Management Systems (12 Marks) — Bus & Route (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Route (
    Routeno INT PRIMARY KEY,
    source VARCHAR(50),
    destination VARCHAR(50),
    no_of_stations INT
);

-- Create Bus table
CREATE TABLE Bus (
    Busno INT PRIMARY KEY,
    capacity INT NOT NULL,    -- Constraint: cannot be null
    depot_name VARCHAR(50),
    Routeno INT,              -- Foreign key (Many buses → One route)
    FOREIGN KEY (Routeno) REFERENCES Route(Routeno)
);

-- Insert some sample records into Route
INSERT INTO Route VALUES (41, 'Pune Station', 'Kothrud', 12);
INSERT INTO Route VALUES (42, 'Swargate', 'Nigdi', 15);
INSERT INTO Route VALUES (43, 'Pimpri', 'Hadapsar', 18);
INSERT INTO Route VALUES (44, 'Katraj', 'Shivaji Nagar', 2);

-- Insert some sample records into Bus
INSERT INTO Bus VALUES (201, 50, 'Swargate', 41);
INSERT INTO Bus VALUES (202, 45, 'Nigdi', 42);
INSERT INTO Bus VALUES (203, 60, 'Swargate', 43);
INSERT INTO Bus VALUES (204, 40, 'Kothrud', 42);
INSERT INTO Bus VALUES (205, 55, 'Pune Station', 44);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i) List all buses at depot “Swargate”
SELECT * FROM Bus WHERE depot_name = 'Swargate';

ii) Delete Bus whose Bus number is “204”
DELETE FROM Bus WHERE Busno = 204;

iii) List all buses on route number 41
SELECT * FROM Bus WHERE Routeno = 41;



iv) List the route details having number of stations greater than 10
SELECT * FROM Route WHERE no_of_stations > 10;

v) List all routes starting from “Pune station”
SELECT * FROM Route WHERE source = 'Pune Station';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Delete buses on routes with < 3 stations
DELETE FROM Bus
WHERE Routeno IN (
    SELECT Routeno FROM Route WHERE no_of_stations < 3
);

OR
Option 2: Create a view (Pimpri → Hadapsar)
CREATE VIEW Pimpri_Hadapsar_Buses AS
SELECT b.Busno, b.capacity, b.depot_name, r.Routeno, r.source,
r.destination, r.no_of_stations
FROM Bus b
JOIN Route r ON b.Routeno = r.Routeno
WHERE r.source = 'Pimpri' AND r.destination = 'Hadapsar';
```

SLIP 15 [Rank #12 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Perform the following operations using dynamic implementation of Queue
i. Insert n elements in a queue.
ii. Delete the element from the queue and display it. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables (primary keys are underlined).
Game (gname, noofplayers, coachname, captain_name)
Player (pno, pname)
There exists a one-to-many relationship between Game and Player.
Constraint captain_name should be uppercase.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List the names of players playing “hockey”.
ii. Give the average number of players.
iii. Delete the records of players playing ‘kho kho’.
iv. List names of players not playing “cricket”.
v. Update the coach name from “Dhoni” to “Tendulkar” for
game “Cricket”.
C. Write a query to display the game details with maximum number of players. [2 marks]
OR
C. Create a view to print game-wise list of Players along with their game name
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Insertion Sort on Random Array: The playing cards sorting technique. For each element at index `i`, we pick it up into `key`. Then we compare backwards with elements at `j = i - 1`. As long as `arr[j] > key`, we slide `arr[j]` one step to the right (`arr[j+1] = arr[j]`). When we find a smaller number or hit the front, we drop `key` into `arr[j+1]`. Much simpler than dynamic stack implementations!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Gaming Tournament. Players register under specific Video Games (One-to-Many). Player references `Game(game_id)`. For queries: Query (i) lists players who scored more than 1000 points (`WHERE score > 1000`). Query (ii) finds the top scorer for each game. Query (iii) updates game release date.

🧠 **Mental Formula & Memory Trick:** The Card Player's Hand: Pick card i (key = arr[i]). Compare with cards to its left (j = i - 1). While j >= 0 and arr[j] > key, slide them right: arr[j+1] = arr[j]. Finally, drop key in empty slot: arr[j+1] = key.

💡 **Winning Exam Strategy:** Option A avoids dynamic linked queue. Insertion sort is intuitive (card-sorting simulation) and needs only 25 lines.

💻 Section I: Data Structures (13 Marks) — Insertion Sort on Random Array

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, key, j;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    srand(time(0));

    printf("Random Array: ");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    // [MEMORY TRICK]: Hold current card, slide bigger cards right, insert card
    for (int i = 1; i < n; i++) {
        key = arr[i]; // Pick card
        j = i - 1;

        // Shift elements greater than key to the right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key; // Place card in slot
    }

    printf("Sorted Array (Insertion Sort): ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 5
Random Array: 77 14 3 62 49
Sorted Array (Insertion Sort): 3 14 49 62 77
```

🗄️ Section II: Database Management Systems (12 Marks) — Game & Player (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Game (
    gname VARCHAR(30) PRIMARY KEY,
    noofplayers INT,
    coachname VARCHAR(30),
    captain_name VARCHAR(30) CHECK (captain_name =
UPPER(captain_name))  -- must be uppercase
);

-- Create Player table

CREATE TABLE Player (
    pno INT PRIMARY KEY,
    pname VARCHAR(30),
    gname VARCHAR(30),
    FOREIGN KEY (gname) REFERENCES Game(gname)
);

-- Insert sample games
INSERT INTO Game VALUES ('Cricket', 11, 'Dhoni', 'KOHLI');
INSERT INTO Game VALUES ('Hockey', 11, 'Singh', 'RAHUL');
INSERT INTO Game VALUES ('Kho Kho', 12, 'Sharma', 'AMAN');
INSERT INTO Game VALUES ('Football', 11, 'Jose', 'ROHIT');

-- Insert sample players
INSERT INTO Player VALUES (1, 'Amit', 'Cricket');
INSERT INTO Player VALUES (2, 'Ravi', 'Cricket');
INSERT INTO Player VALUES (3, 'Sohan', 'Hockey');
INSERT INTO Player VALUES (4, 'Mohan', 'Hockey');
INSERT INTO Player VALUES (5, 'Raj', 'Kho Kho');
INSERT INTO Player VALUES (6, 'Anil', 'Football');
INSERT INTO Player VALUES (7, 'Sunil', 'Football');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. List the names of players playing "hockey"
SELECT pname FROM Player
WHERE gname = 'Hockey';


ii. Give the average number of players
SELECT AVG(noofplayers) AS avg_players


FROM Game;


iii. Delete the records of players playing 'kho kho'
DELETE FROM Player
WHERE gname = 'Kho Kho';


iv. List names of players not playing "cricket"
SELECT pname FROM Player
WHERE gname <> 'Cricket';


v. Update the coach name from "Dhoni" to "Tendulkar" for game "Cricket"
UPDATE Game
SET coachname = 'Tendulkar'
WHERE gname = 'Cricket' AND coachname = 'Dhoni';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Display game details with maximum number of players
SELECT * FROM Game
WHERE noofplayers = (SELECT MAX(noofplayers) FROM Game);


Option 2: Create a view to print game-wise list of players
CREATE VIEW GameWisePlayers AS
SELECT g.gname, p.pname
FROM Game g
JOIN Player p ON g.gname = p.gname;
```

SLIP 17 [Rank #13 of 25]

TIER 2 ⏱️ 15-18 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using recursive Quick sort algorithm. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Perform the following operations using implementation of stack (Static or dynamic)
i. Accept n integers from user and push it into the stack.
ii. Reverses a string of characters. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (Primary keys are underlined)
Property (Pno, area_Sqft, location, city)
Owner (oname, address, phone)
The relationship is as follows:
Property – Owner are related with Many to One relationship.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Give the list of all properties in Nashik city.
ii. Give the names of owners whose property is greater than 2000sq feet.
iii. Find the number of properties owned by “Mr.Rahane”.
iv. Count all properties of “Pune” city.
v. Delete all properties in “Pune” city owned by “Mr.Baravkar”.
C. Write a query to find the names of all owners who own a property in Pune city, located
in ‘Kothrud’ area and which has an area greater than 10000sqft.
OR
C. Create a view to find the owner wise list of properties.
[2 marks]
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Stack Push & Reverse: Accepts n numbers from the user and pushes them into an array stack. Popping and displaying them automatically reverses the sequence because LIFO inverts entry order. Compact, 25-line program that compiles with 0 errors.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Real Estate Registry. Properties and Owners (Many-to-Many). Connected via `Property_Owner` bridge table with `percentage_share`. For queries: Query (i) lists properties located in 'Pune'. Query (ii) filters properties with valuation above 50,00,000. Query (iii) updates property market valuation.

🧠 **Mental Formula & Memory Trick:** Dual Stack Rule: Two simple stacks in one file: integer array for numbers, char array for letters. Push with ++top, pop with top--. Avoids recursive Quick Sort.

💡 **Winning Exam Strategy:** Option B avoids recursive Quick Sort. Static stack array easily handles both integer pushes and character string reversal in under 35 lines.

💻 Section I: Data Structures (13 Marks) — Static Stack (Push Integers & Reverse String)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <string.h>
#define MAX 100

// Stack 1: Integers
int stack_int[MAX]; int top_int = -1;
void push_int(int val) { stack_int[++top_int] = val; }
int pop_int() { return stack_int[top_int--]; }

// Stack 2: Characters
char stack_char[MAX]; int top_char = -1;
void push_char(char c) { stack_char[++top_char] = c; }
char pop_char() { return stack_char[top_char--]; }

int main() {
    int n, val;
    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push_int(val);
    }
    printf("Popped integers: ");
    while (top_int != -1) printf("%d ", pop_int());
    printf("\n");

    char str[MAX];
    printf("\nEnter string to reverse: ");
    scanf(" %[^\n]", str);
    for (int i = 0; i < strlen(str); i++) push_char(str[i]);

    printf("Reversed string: ");
    while (top_char != -1) printf("%c", pop_char());
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of integers: 3
Enter 3 integers: 10 20 30
Popped integers: 30 20 10

Enter string to reverse: Examination
Reversed string: noitanimaxE
```

🗄️ Section II: Database Management Systems (12 Marks) — Property & Owner (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Owner (


    oname VARCHAR(50) PRIMARY KEY,
    address VARCHAR(100),
    phone VARCHAR(15)
);

-- Create Property table (Many properties → One owner)
CREATE TABLE Property (
    pno INT PRIMARY KEY,
    area_sqft INT,
    location VARCHAR(50),
    city VARCHAR(50),
    oname VARCHAR(50),
    FOREIGN KEY (oname) REFERENCES Owner(oname)
);

-- Insert Owners
INSERT INTO Owner VALUES ('Mr.Rahane', 'Nashik', '9876543210');
INSERT INTO Owner VALUES ('Mr.Baravkar', 'Pune', '9123456789');
INSERT INTO Owner VALUES ('Mr.Kulkarni', 'Mumbai', '9988776655');



-- Insert Properties
INSERT INTO Property VALUES (1, 2500, 'Gangapur Road', 'Nashik',
'Mr.Rahane');
INSERT INTO Property VALUES (2, 1800, 'Kothrud', 'Pune',
'Mr.Baravkar');
INSERT INTO Property VALUES (3, 12000, 'Kothrud', 'Pune',
'Mr.Kulkarni');
INSERT INTO Property VALUES (4, 3000, 'Deccan', 'Pune', 'Mr.Rahane');
INSERT INTO Property VALUES (5, 900, 'Dadar', 'Mumbai', 'Mr.Kulkarni');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. List of all properties in Nashik city
SELECT * FROM Property
WHERE city = 'Nashik';


ii. Owners whose property > 2000 sqft
SELECT oname


FROM Property
WHERE area_sqft > 2000;




iii. Number of properties owned by "Mr.Rahane"
SELECT COUNT(*) AS property_count
FROM Property
WHERE oname = 'Mr.Rahane';


iv. Count all properties of Pune city
SELECT COUNT(*) AS pune_properties
FROM Property
WHERE city = 'Pune';




v. Delete all properties in Pune owned by "Mr.Baravkar"
DELETE FROM Property
WHERE city = 'Pune' AND oname = 'Mr.Baravkar';


Option 1 (Simple Version)
Find owners who have property in Pune city, located in Kothrud, and area
greater than 10000 sqft:
SELECT DISTINCT oname
FROM Property
WHERE city = 'Pune'
  AND location = 'Kothrud'
  AND area_sqft > 10000;
```

Part C: Advanced Query / View [2 Marks]

```
Option 2 (View for Owner-wise property list)
CREATE VIEW OwnerWiseProperties AS
SELECT oname, pno, area_sqft, location, city


FROM Property;
```

SLIP 2 [Rank #14 of 25]

TIER 2 ⏱️ 15-18 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q.1) Write a C program to implement static Implementation of Queue of integers with following operation:

- Initialize()
- Insert()
- Delete()
- Isempty()
- Display() 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q.1)
Write a menu driven program for the following operations on singly linked list.
i. Create and display a linked list of n nodes.
ii. Search a particular element from a linked list. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database( primary keys are underlined).
Employee(eno,ename,designation,salary)
Department(dno,dname,location)
The relationship is as follows: Employee-Department: many-to-one.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Give a 5% raise in salary to all the employees.
ii. Display average salary of an employee.
iii. List the details of all the departments located at ‘Nashik’city.
iv. Display the details of employees whose names ends with an alphabet ‘a’.
v. Display the location of ‘HR’ department.
C. Write a query to list the details of employees who do not work in any of the departments located
at ‘Delhi’.
[2 marks]
OR
C. Create a view to list the names of employees whose salary is greater than
all the employees working in ‘HR’ department.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** A Queue is a 'First In, First Out' (FIFO) line — imagine standing in line at a movie ticket counter. The first person to arrive is the first person served. We track two pointers: `front` (where people leave after service) and `rear` (where new people join the line). Both start at -1. When you insert the very first number, set `front = 0`, then move `rear++`. When deleting, increment `front++`. If `front > rear`, the queue is empty!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Company HR. Multiple Employees work in one Department (Many-to-One). Department is the parent table (`dno PRIMARY KEY`), and Employee references it with `dno REFERENCES Department(dno)`. For queries: Query (i) raises everyone's salary by 5% using `UPDATE Employee SET salary = salary * 1.05`. Query (ii) computes average salary using `SELECT AVG(salary) FROM Employee`. Query (iii) finds departments located in Nashik using `WHERE location = 'Nashik'`.

🧠 **Mental Formula & Memory Trick:** The Ticket Counter Rule: Two pointers: 'front' (who leaves) and 'rear' (who joins). Both start at -1. Insert moves rear forward: q[++rear] = val. Delete moves front forward: q[front++]. Queue is empty if front == -1 or front > rear.

> 💡 **Winning Exam Strategy:** Static Queue uses a simple array and front/rear indices (35 lines). Much faster and safer to write in exam than a dynamic singly linked list.

💻 Section I: Data Structures (13 Marks) — Static Queue (Initialize, Insert, Delete, isEmpty, Display)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

// [MEMORY TRICK]: Both pointers start at -1
int queue[MAX];
int front = -1, rear = -1;

void initialize() {
    front = -1;
    rear = -1;
}

// [TRICK]: Empty if uninitialized (-1) OR front has passed rear
int isEmpty() {
    return (front == -1 || front > rear);
}

// [TRICK]: Rear welcomes newcomers: ++rear
void insert(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow!\n");
        return;
    }
    if (front == -1) front = 0; // First person sets front to 0
    queue[++rear] = value;
    printf("Inserted: %d\n", value);
}

// [TRICK]: Front serves and leaves: front++
void delete() {
    if (isEmpty()) {
        printf("Queue is Empty!\n");
        return;
    }
    printf("Deleted: %d\n", queue[front++]);
}

void display() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue contents: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    initialize();
    insert(10);
    insert(20);
    insert(30);
    display();
    delete();
    display();
    return 0;
}
```

Expected Terminal Output:

```
Inserted: 10
Inserted: 20
Inserted: 30
Queue contents: 10 20 30
Deleted: 10
Queue contents: 20 30
```

🗄️ Section II: Database Management Systems (12 Marks) — Employee & Department (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Department (
    dno INT PRIMARY KEY,
    dname VARCHAR(50),
    location VARCHAR(50)
);

-- Create Employee table (many-to-one: each employee belongs to one department)
CREATE TABLE Employee (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),
    designation VARCHAR(50),
    salary NUMERIC(10,2),
    dno INT,
    FOREIGN KEY (dno) REFERENCES Department(dno)
);

-- Insert sample Department records
INSERT INTO Department VALUES
(1, 'HR', 'Delhi'),
(2, 'Finance', 'Nashik'),
(3, 'IT', 'Mumbai'),
(4, 'Admin', 'Nashik');

-- Insert sample Employee records
INSERT INTO Employee VALUES


(101, 'Ravi', 'Manager', 50000, 1),
(102, 'Meena', 'Clerk', 25000, 2),
(103, 'Aruna', 'Analyst', 30000, 2),
(104, 'Kiran', 'Developer', 45000, 3),
(105, 'Sonia', 'HR Executive', 28000, 1);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Give a 5% raise in salary to all employees
UPDATE Employee
SET salary = salary * 1.05;

ii. Display average salary of employees
SELECT AVG(salary) AS avg_salary
FROM Employee;

iii. List details of all departments located at 'Nashik'
SELECT *
FROM Department
WHERE location = 'Nashik';

iv. Display employees whose names end with 'a'
SELECT *
FROM Employee
WHERE ename LIKE '%a';

v. Display the location of 'HR' department
SELECT location
FROM Department
WHERE dname = 'HR';
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice


1. Employees not working in departments located at 'Delhi'
SELECT *
FROM Employee
WHERE dno NOT IN (
    SELECT dno
    FROM Department
    WHERE location = 'Delhi'
);


2. Create a view: employees with salary greater than all employees in HR
CREATE VIEW HighSalaryEmployees AS
SELECT ename, salary
FROM Employee
WHERE salary > ALL (
    SELECT salary
    FROM Employee E
    JOIN Department D ON E.dno = D.dno
    WHERE D.dname = 'HR'
);

OR

CREATE VIEW HighSalaryEmployees AS
SELECT ename, salary
FROM Employee
WHERE salary > ALL (
    SELECT salary
    FROM Employee
    WHERE dno = (
        SELECT dno
        FROM Department
        WHERE dname = 'HR'
    )
);
```

SLIP 10 [Rank #15 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Create random array of n integers and sort the array in ascending order by using recursive Merge sort algorithm ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Write a C program to implement Static implementation of stack of integers with following operation.
a. Initialize()
b. Push()
c. Pop()
d. Isempty()
e. Isfull()
f. Peek() 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables ( primary keys are underlined).
Account (acctno ,acct_type, balance, branch_name)
Customer (custno, cust_name, cust_city)
Relationships: Customer-Account: One to Many.
Constraints: acct_type should be “saving” or “current”and balance is greater than
zero.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Display information of all saving accounts having balance greater than 5000.
ii. Count customers in city “Pune”.
iii. Find the total balance at branch “M.G.Road”.
iv. Delete the record whose cust_name is “Ramnath”.
v. Change city of customer “Sanjay” to “Pune”
C. Write a query to find the number of account holders for each city.
[2 marks]
OR
C. Create a view to print the account details along with customer and their branch details.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Stack with Full Modular Suite: Implements all textbook stack primitives: `init()` (resets `top = -1`), `isFull()` (returns true if `top == MAX - 1`), `isEmpty()` (returns true if `top == -1`), `push(x)` (adds item), `pop()` (removes item), and `peek()` (looks at current top element without deleting it). Fast to write, clean to present, and guaranteed zero segfaults.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Retail Banking. A Customer holds one or more Accounts (One-to-Many). Account references `Customer(cno)` and has a minimum balance rule: `CHECK (balance >= 500)`. For queries: Query (i) finds high-wealth accounts with `WHERE balance > 50000`. Query (ii) sums all money deposited in the bank with `SELECT SUM(balance) FROM Account`. Query (iii) counts accounts per branch using `GROUP BY branch`.

🧠 **Mental Formula & Memory Trick:** The Vending Stack: 1. init: top = -1. 2. isEmpty: top == -1. 3. isFull: top == MAX - 1. 4. push: stack[++top] = val. 5. pop: stack[top--]. 6. peek: stack[top] (look without popping).

💡 **Winning Exam Strategy:** Option B avoids complex recursive Merge Sort (auxiliary array, mid calculation, merge procedure). Static stack is purely sequential and foolproof.

💻 Section I: Data Structures (13 Marks) — Static Stack Operations (Init, Push, Pop, isEmpty, isFull, Peek)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void initialize() { top = -1; }
int isEmpty() { return top == -1; }
int isFull() { return top == MAX - 1; }

void push(int val) {
    if (isFull()) {
        printf("Stack Overflow!\n");
        return;
    }
    stack[++top] = val; // Move up, insert
    printf("Pushed: %d\n", val);
}

int pop() {
    if (isEmpty()) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--]; // Return, move down
}

int peek() {
    if (isEmpty()) return -1;
    return stack[top]; // Inspect top without moving
}

int main() {
    initialize();
    push(10);
    push(20);
    push(30);
    printf("Top element (Peek): %d\n", peek());
    printf("Popped element: %d\n", pop());
    printf("Popped element: %d\n", pop());
    printf("Is stack empty? %s\n", isEmpty() ? "Yes" : "No");
    return 0;
}
```

Expected Terminal Output:

```
Pushed: 10
Pushed: 20
Pushed: 30
Top element (Peek): 30
Popped element: 30
Popped element: 20
Is stack empty? No
```

🗄️ Section II: Database Management Systems (12 Marks) — Customer & Account (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Customer (
    custno INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
);

-- Account table
CREATE TABLE Account (
    acctno INT PRIMARY KEY,
    acct_type VARCHAR(10) CHECK (acct_type IN ('saving','current')),
    balance NUMERIC(12,2) CHECK (balance > 0),
    branch_name VARCHAR(50),
    custno INT,
    FOREIGN KEY (custno) REFERENCES Customer(custno)
);

-- Insert Customers
INSERT INTO Customer VALUES
(1, 'Ravi', 'Pune'),
(2, 'Meena', 'Mumbai'),
(3, 'Sanjay', 'Delhi'),
(4, 'Ramnath', 'Nashik');



-- Insert Accounts
INSERT INTO Account VALUES
(101, 'saving', 7000, 'M.G.Road', 1),
(102, 'current', 12000, 'Deccan', 2),
(103, 'saving', 3000, 'M.G.Road', 3),
(104, 'saving', 15000, 'Camp', 1),
(105, 'current', 9000, 'M.G.Road', 4);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```sql
B. Queries (any 3)
i. All saving accounts with balance > 5000
SELECT *
FROM Account
WHERE acct_type = 'saving'
  AND balance > 5000;

ii. Count customers in Pune
SELECT COUNT(*) AS pune_customers
FROM Customer
WHERE cust_city = 'Pune';

iii. Total balance at branch M.G.Road
SELECT SUM(balance) AS total_balance
FROM Account
WHERE branch_name = 'M.G.Road';

iv. Delete customer named Ramnath
DELETE FROM Customer
WHERE cust_name = 'Ramnath';

v. Change city of customer Sanjay → Pune
UPDATE Customer
SET cust_city = 'Pune'
WHERE cust_name = 'Sanjay';
```

Part C: Advanced Query / View [2 Marks]

```
C. any one
1. Number of account holders per city
SELECT C.cust_city, COUNT(A.acctno) AS total_accounts
FROM Customer C, Account A
WHERE C.custno = A.custno
GROUP BY C.cust_city;


2. View: Account details with customer + branch
CREATE VIEW AccountCustomerView AS
SELECT A.acctno, A.acct_type, A.balance, A.branch_name,
       C.cust_name, C.cust_city
FROM Account A, Customer C
WHERE A.custno = C.custno;
```

SLIP 20 [Rank #16 of 25]

TIER 2 ⏱️ 15-18 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Read the data from the 'employee.txt' file and sort on age using Merge sort. Write the sorted data to another file 'sortedemponage.txt'. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Write a C program to implement Static implementation of stack of integers with following operation.
a. Initialize()
b. Push()
c. Pop()
d. Isempty()
e. Isfull()
f. Peek() 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (Primary keys are underlined)
Customer (cust_no, cust_name, cust_city)
Account (acc_no, acc_type , balance)
Account type can be either “Saving” or “Current”
There exists a One-to-Many relationship between Customer and Account.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B .Execute the following queries in PostGreSQL (any 3) [6 marks]
i. Find the names of all Customers staying in “Delhi”.
ii. Give the total balance of all Saving accounts.
iii. Find the maximum balance of an account.
iv. Increase the balance of account no 101123 by Rs 10,000/-
v. Add column “AADHAR number” to customer table.
C.Write a query to find the names and phone numbers of all Customers of Current
accounts, having balance greater than 10,00,000/-
OR
C.Create a View to list details of all customers having saving account and not staying in
city” Ahilyanagar”.
[2 marks]
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Stack Full Suite: Complete array stack implementation with `init()`, `push()`, `pop()`, `peek()`, `isEmpty()`, and `isFull()`. Avoids reading employee files with binary search and gives the examiner a textbook-perfect stack program.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Customer & Account (One-to-Many). Same bank database as Slip 10. Customer has accounts with `balance >= 500` constraint. For queries: Query (i) finds accounts with balance over 50,00,000. Query (ii) computes sum of all bank balances. Query (iii) counts accounts per branch.

🧠 **Mental Formula & Memory Trick:** The Complete Stack Checklist: 1. init: top = -1. 2. isFull: top == MAX-1. 3. isEmpty: top == -1. 4. push: s[++top] = x. 5. pop: s[top--]. 6. peek: s[top].

💡 **Winning Exam Strategy:** Option B avoids Merge Sort and file output (`sortedemponage.txt`). Array stack is 35 lines, completely avoiding recursion and file writing.

💻 Section I: Data Structures (13 Marks) — Static Stack Operations (Init, Push, Pop, isEmpty, isFull, Peek)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void initialize() { top = -1; }
int isEmpty() { return top == -1; }
int isFull() { return top == MAX - 1; }

void push(int val) {
    if (isFull()) {
        printf("Stack Overflow!\n");
        return;
    }
    stack[++top] = val;
    printf("Pushed: %d\n", val);
}

int pop() {
    if (isEmpty()) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--];
}

int peek() {
    if (isEmpty()) return -1;
    return stack[top];
}

int main() {
    initialize();
    push(100);
    push(200);
    push(300);
    printf("Peek element: %d\n", peek());
    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());
    printf("Is empty? %s\n", isEmpty() ? "Yes" : "No");
    return 0;
}
```

Expected Terminal Output:

```
Pushed: 100
Pushed: 200
Pushed: 300
Peek element: 300
Popped: 300
Popped: 200
Is empty? No
```

🗄️ Section II: Database Management Systems (12 Marks) — Customer & Account (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Customer (


    cust_no INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
);

-- Create Account table
CREATE TABLE Account (
    acc_no INT PRIMARY KEY,
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving','Current')),
    balance NUMERIC CHECK (balance >= 0),
    cust_no INT,
    FOREIGN KEY (cust_no) REFERENCES Customer(cust_no)
);

-- Insert sample Customers
INSERT INTO Customer VALUES (1, 'Amit', 'Delhi');
INSERT INTO Customer VALUES (2, 'Ravi', 'Mumbai');
INSERT INTO Customer VALUES (3, 'Sneha', 'Delhi');
INSERT INTO Customer VALUES (4, 'Meera', 'Ahilyanagar');

-- Insert sample Accounts
INSERT INTO Account VALUES (101123, 'Saving', 500000, 1);
INSERT INTO Account VALUES (101124, 'Current', 1500000, 2);
INSERT INTO Account VALUES (101125, 'Saving', 250000, 3);
INSERT INTO Account VALUES (101126, 'Current', 1200000, 4);
INSERT INTO Account VALUES (101127, 'Saving', 700000, 2);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. Find the names of all Customers staying in "Delhi"
SELECT cust_name FROM Customer
WHERE cust_city = 'Delhi';


ii. Give the total balance of all Saving accounts
SELECT SUM(balance) AS total_saving_balance
FROM Account
WHERE acc_type = 'Saving';


iii. Find the maximum balance of an account


SELECT MAX(balance) AS max_balance
FROM Account;


iv. Increase the balance of account no 101123 by Rs 10,000/-
UPDATE Account
SET balance = balance + 10000
WHERE acc_no = 101123;


v. Add column "AADHAR number" to customer table
ALTER TABLE Customer
ADD COLUMN aadhar_no VARCHAR(12);
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Customers of Current accounts with balance > 10,00,000
SELECT c.cust_name, c.cust_city
FROM Customer c
JOIN Account a ON c.cust_no = a.cust_no
WHERE a.acc_type = 'Current' AND a.balance > 1000000;


Option 2: Create a view for customers having Saving account and not in
“Ahilyanagar”
CREATE VIEW SavingCustomers AS
SELECT DISTINCT c.cust_no, c.cust_name, c.cust_city
FROM Customer c
JOIN Account a ON c.cust_no = a.cust_no
WHERE a.acc_type = 'Saving'
  AND c.cust_city <> 'Ahilyanagar';
```

SLIP 21 [Rank #17 of 25]

TIER 2 ⏱️ 15-18 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Perform the following operations using static implementation of Queue
i. Create a queue of n integers.
ii. Delete the element from the queue and display updated queue. 👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q1) Write a menu driven program for the following operations on singly linked list
i.
Insert an element in a linked list in a particular position.
ii. Search an element in a linked list. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (Primary keys are underlined)
Bus (bno, capacity, depot_name)
Driver (dno, driver_name, license_no, address, age, salary)
The relationship is as follows:
Bus and Driver are related with Many to Many relationships with the descriptive
attribute Date_of_duty.
A. Create the above database in PostGreSQL and insert sufficient records.[4 marks]
B. Execute the following queries in PostGreSQL (any 3) [6 marks]
i. Find the number of buses having capacity more than 20.
ii. Count number of drivers who did duty on “15/01/2017”
iii. Display the license number of all drivers whose name starts with ‘S’
iv. Calculate the average salary of a driver.
v. Find all details of driver named “Swapnil”
C. Write a query to find the names of all drivers who were driving buses
from “Kothrud Depot” on 12/2/2009.
[2 marks]
OR
C. Create a view to find all details of buses which are driven by drivers whose age is
less than 25 years.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Queue (Create, Delete, Display): We accept n elements into an array-based queue using `rear++`. We delete the front element by advancing `front++`. Then we display the remaining items from index `front` to `rear`. Straightforward FIFO demonstration in under 30 lines.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Transit Bus & Driver (Many-to-Many). Bus drivers assigned to buses on various dates. Bridge table `Bus_Driver` contains `bus_no`, `driver_id`, and `shift_date`. For queries: Query (i) lists buses driven by a specific driver. Query (ii) counts drivers on duty today. Query (iii) updates driver license details.

🧠 **Mental Formula & Memory Trick:** Create Queue of n: Call insert() in a simple for loop. Delete once (front++). Display updated queue from front to rear. No linked list node shifting!

💡 **Winning Exam Strategy:** Option A avoids Singly Linked List insertion at arbitrary positions. Queue deletion and display using an array takes under 35 lines.

💻 Section I: Data Structures (13 Marks) — Static Queue (Create Queue of n, Delete, Display Updated) ✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

void insert(int val) {
    if (rear == MAX - 1) {
        printf("Queue Overflow!\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = val;
}

void delete() {
    if (front == -1 || front > rear) {
        printf("Queue is empty!\n");
        return;
    }
    printf("Deleted element: %d\n", queue[front++]);
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue contents: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    int n, val;
    printf("Enter number of elements for queue: ");
    if (scanf("%d", &n) != 1) return 0;

    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insert(val);
    }

    display();
    printf("Deleting one element...\n");
    delete();
    display();
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements for queue: 4
Enter 4 integers: 10 20 30 40
Queue contents: 10 20 30 40
Deleting one element...
Deleted element: 10
Queue contents: 20 30 40
```

🗄️ Section II: Database Management Systems (12 Marks) — Bus & Driver (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Bus (
    bno INT PRIMARY KEY,
    capacity INT,
    depot_name VARCHAR(50)
);

-- Create Driver table
CREATE TABLE Driver (
    dno INT PRIMARY KEY,
    driver_name VARCHAR(50),
    license_no VARCHAR(20),
    address VARCHAR(50),
    age INT,
    salary NUMERIC
);

-- Create Junction table with descriptive attribute Date_of_duty
CREATE TABLE BusDriver (
    bno INT,
    dno INT,
    date_of_duty DATE,
    PRIMARY KEY (bno, dno, date_of_duty),
    FOREIGN KEY (bno) REFERENCES Bus(bno),
    FOREIGN KEY (dno) REFERENCES Driver(dno)


);

-- Insert sample Buses
INSERT INTO Bus VALUES (101, 30, 'Kothrud Depot');
INSERT INTO Bus VALUES (102, 40, 'Swargate Depot');
INSERT INTO Bus VALUES (103, 25, 'Kothrud Depot');

-- Insert sample Drivers
INSERT INTO Driver VALUES (1, 'Swapnil', 'LIC123', 'Pune', 28, 30000);
INSERT INTO Driver VALUES (2, 'Sohan', 'LIC124', 'Mumbai', 24, 28000);
INSERT INTO Driver VALUES (3, 'Sunil', 'LIC125', 'Pune', 26, 32000);

-- Insert sample duties
INSERT INTO BusDriver VALUES (101, 1, '2009-02-12');
INSERT INTO BusDriver VALUES (101, 2, '2009-02-12');
INSERT INTO BusDriver VALUES (102, 3, '2017-01-15');
INSERT INTO BusDriver VALUES (103, 2, '2017-01-15');
INSERT INTO BusDriver VALUES (103, 1, '2009-02-12');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. Number of buses having capacity more than 20
SELECT COUNT(*) AS bus_count
FROM Bus
WHERE capacity > 20;


ii. Count number of drivers who did duty on "15/01/2017"
SELECT COUNT(DISTINCT dno) AS driver_count
FROM BusDriver
WHERE date_of_duty = '2017-01-15';


iii. Display license number of all drivers whose name starts with 'S'
SELECT license_no
FROM Driver
WHERE driver_name LIKE 'S%';




iv. Calculate average salary of a driver
SELECT AVG(salary) AS avg_salary
FROM Driver;


v. Find all details of driver named "Swapnil"
SELECT *
FROM Driver
WHERE driver_name = 'Swapnil';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Names of all drivers driving buses from "Kothrud Depot" on 12/2/2009
SELECT DISTINCT d.driver_name
FROM Driver d
JOIN BusDriver bd ON d.dno = bd.dno
JOIN Bus b ON bd.bno = b.bno
WHERE b.depot_name = 'Kothrud Depot'
  AND bd.date_of_duty = '2009-02-12';


Option 2: View for all buses driven by drivers aged less than 25 years
CREATE VIEW YoungDriverBuses AS
SELECT DISTINCT b.*
FROM Bus b
JOIN BusDriver bd ON b.bno = bd.bno
JOIN Driver d ON bd.dno = d.dno
WHERE d.age < 25;


OR

CREATE VIEW YoungDriverBuses AS
SELECT *
FROM Bus
WHERE bno IN (
    SELECT bno
    FROM BusDriver bd
    JOIN Driver d ON bd.dno = d.dno


    WHERE d.age < 25
);
```

SLIP 23 [Rank #18 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Write a menu driven program for the following operations on doubly linked list.
i. Create and display linked list.
ii. Reverse a linked list and display it. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Perform the following operations using static implementation of stack
i. Accept n integers from user and push it into the stack.
ii. Pop the element from the stack and display it (Use isEmpty()) 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (Primary keys are underlined)
Customer (cno, cust_name , cust_city)
Account (ano,acc_type , balance)
There exists a One to Many relationships between Customer and Account.
Constraint:Account type Should be either “Saving” or “Current”
A.Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B.Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Count the number of customers whose name contains substring “in”.
ii. Give the average balance of all Current accounts.
iii. Display details of accounts in sorted order of balance.
iv. Increase the balance of all accounts of customer named as “Sahil” by
Rs 10000.
v. Add column “phone_no” to customer table.
C.Write a query to list the city-wise customer names having minimum balance. [2 marks]
OR
C. Create a view to list details of customers in Pune having maximum balance in saving
account.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Stack Push & Pop with isEmpty: Quick and clean array stack. User inputs n integers which get pushed into `stack[top]`. Then a `while(!isEmpty())` loop pops and displays each element until the stack is exhausted.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Customer & Account (One-to-Many). Banking schema. Queries list accounts meeting minimum balance thresholds, calculate branch totals, and list accounts by city.

🧠 **Mental Formula & Memory Trick:** The Minimalist Stack: Push: stack[++top] = val. Pop: stack[top--]. isEmpty: top == -1. Avoids Doubly Linked List completely.

💡 **Winning Exam Strategy:** Option B avoids Doubly Linked List (create, display, reverse). Array stack push & pop takes only 25 lines with zero pointers.

💻 Section I: Data Structures (13 Marks) — Static Stack (Push & Pop with isEmpty)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

int isEmpty() { return top == -1; }

void push(int val) {
    if (top < MAX - 1) stack[++top] = val;
}

int pop() {
    if (isEmpty()) return -1;
    return stack[top--];
}

int main() {
    int n, val;
    printf("Enter number of integers: ");
    if (scanf("%d", &n) != 1) return 0;

    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push(val);
    }

    printf("Popping elements using isEmpty():\n");
    while (!isEmpty()) printf("%d ", pop());
    printf("\n");
    return 0;
}
```

Expected Terminal Output:

```
Enter number of integers: 3
Enter 3 integers: 5 15 25
Popping elements using isEmpty():
25 15 5
```

🗄️ Section II: Database Management Systems (12 Marks) — Customer & Account (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Customer (
    cno INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
);

-- Create Account table (One customer → Many accounts)
CREATE TABLE Account (
    ano INT PRIMARY KEY,
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving','Current')),
    balance NUMERIC CHECK (balance >= 0),
    cno INT,
    FOREIGN KEY (cno) REFERENCES Customer(cno)
);

-- Insert sample Customers
INSERT INTO Customer VALUES (1, 'Sahil', 'Pune');
INSERT INTO Customer VALUES (2, 'Rina', 'Mumbai');
INSERT INTO Customer VALUES (3, 'Kiran', 'Pune');
INSERT INTO Customer VALUES (4, 'Meera', 'Delhi');
INSERT INTO Customer VALUES (5, 'Ankit', 'Pune');

-- Insert sample Accounts
INSERT INTO Account VALUES (101, 'Saving', 50000, 1);
INSERT INTO Account VALUES (102, 'Current', 150000, 2);
INSERT INTO Account VALUES (103, 'Saving', 25000, 3);
INSERT INTO Account VALUES (104, 'Current', 120000, 4);
INSERT INTO Account VALUES (105, 'Saving', 70000, 5);
INSERT INTO Account VALUES (106, 'Current', 200000, 1);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
i. Count the number of customers whose name contains substring "in"
SELECT COUNT(*) AS customer_count
FROM Customer


WHERE cust_name LIKE '%in%';


ii. Give the average balance of all Current accounts
SELECT AVG(balance) AS avg_current_balance
FROM Account
WHERE acc_type = 'Current';


iii. Display details of accounts in sorted order of balance
SELECT *
FROM Account
ORDER BY balance;


iv. Increase the balance of all accounts of customer named "Sahil" by Rs 10000
UPDATE Account
SET balance = balance + 10000
WHERE cno = (
    SELECT cno FROM Customer WHERE cust_name = 'Sahil'
);


v. Add column "phone_no" to customer table
ALTER TABLE Customer
ADD COLUMN phone_no VARCHAR(15);

Option 1: City-wise customer names having
minimum balance (easy version)
SELECT c.cust_city, c.cust_name, a.balance
FROM Customer c
JOIN Account a ON c.cno = a.cno
WHERE a.balance = (
    SELECT MIN(a2.balance)
    FROM Account a2
    JOIN Customer c2 ON a2.cno = c2.cno
    WHERE c2.cust_city = c.cust_city


);
```

Part C: Advanced Query / View [2 Marks]

```
Option 2: View for customers in Pune with maximum
Saving balance (easy version)

CREATE VIEW PuneMaxSaving AS
SELECT c.cust_name, c.cust_city, a.balance
FROM Customer c
JOIN Account a ON c.cno = a.cno
WHERE c.cust_city = 'Pune' AND a.acc_type = 'Saving'
  AND a.balance = (
      SELECT MAX(balance)
      FROM Account a2
      JOIN Customer c2 ON a2.cno = c2.cno
      WHERE c2.cust_city = 'Pune' AND a2.acc_type = 'Saving'
  );
```

TIER 3: MODERATE

Moderate slips involving Linked Lists, dynamic pointers, or file parsing.

SLIP 22 [Rank #19 of 25]

TIER 3 ⏱️ 22-25 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Read the data from the "employee.txt" file and sort on age using Quick sort and write the sorted data to another file 'sortedemponage.txt'. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Write a menu driven program for the following operations on singly linked list.
i. Insert an element in a linked list in a particular position.
ii. Delete a particular element from a linked list. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database(Primary keys are underlined)
Employee(eno,ename,designation,salary)
Department(dno,dname,location)
The relationship is as follows:
Employee-Department: One to Many.
Constraint: employee salary greater than 5000
A.Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Change designation of employee " Sagar" to Manager.
ii. Display the minimum salary of an employee.
iii. List the details of all the departments located at Nanded city.
iv. Display the details of employees whose names starts with A.
v. List the department number wise count of employees.
C.Write a query to list the department name-wise total salary.
[2 marks]
OR
C. Create a view to display the department details having maximum employees.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Singly Linked List Insert at Position & Delete: Demonstrates pointer manipulation on a chain of nodes. To insert at position k, traverse to node k-1, point `new_node->next = temp->next`, then `temp->next = new_node`. To delete a node, point the preceding node's `next` directly to `temp->next` and `free(temp)`.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Employee & Department (One-to-Many). Same corporate HR database as Slip 2. Department is parent, Employee child. For queries: Query (i) gives 5% raise to employees. Query (ii) computes average salary. Query (iii) lists departments in Nashik.

🧠 **Mental Formula & Memory Trick:** The Bridge Relinker: Insert at pos: walk to pos - 1. newNode->next = temp->next; temp->next = newNode;. Delete value: walk until temp->next->data == val. bypass it: del = temp->next; temp->next = del->next; free(del);.

💡 **Winning Exam Strategy:** Option B avoids file Quick Sort on employee.txt and secondary file writing. Singly linked list pointer traversal is standard in SPPU syllabus.

💻 Section I: Data Structures (13 Marks) — Singly Linked List (Insert at Position, Delete Element)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void insertAtPos(int val, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }
    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        free(newNode);
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteVal(int val) {
    if (head == NULL) return;
    if (head->data == val) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
        printf("Deleted %d\n", val);
        return;
    }
    struct Node* temp = head;
    while (temp->next && temp->next->data != val) temp = temp->next;
    if (temp->next) {
        struct Node* del = temp->next;
        temp->next = del->next;
        free(del);
        printf("Deleted %d\n", val);
    }
}

void display() {
    struct Node* temp = head;
    printf("Linked List: ");
    while (temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    insertAtPos(10, 1);
    insertAtPos(20, 2);
    insertAtPos(15, 2); // Insert 15 at pos 2
    display();
    deleteVal(15);
    display();
    return 0;
}
```

Expected Terminal Output:

```
Linked List: 10 -> 15 -> 20 -> NULL
Deleted 15
Linked List: 10 -> 20 -> NULL
```

🗄️ Section II: Database Management Systems (12 Marks) — Employee & Department (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Department (
    dno INT PRIMARY KEY,
    dname VARCHAR(50),
    location VARCHAR(50)
);

-- Create Employee table (One department → Many employees)
CREATE TABLE Employee (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),
    designation VARCHAR(50),
    salary NUMERIC CHECK (salary > 5000),
    dno INT,
    FOREIGN KEY (dno) REFERENCES Department(dno)
);

-- Insert Departments
INSERT INTO Department VALUES (1, 'HR', 'Nanded');
INSERT INTO Department VALUES (2, 'Finance', 'Pune');


INSERT INTO Department VALUES (3, 'IT', 'Mumbai');

-- Insert Employees
INSERT INTO Employee VALUES (101, 'Sagar', 'Clerk', 12000, 1);
INSERT INTO Employee VALUES (102, 'Amit', 'Analyst', 15000, 2);
INSERT INTO Employee VALUES (103, 'Anil', 'Developer', 18000, 3);
INSERT INTO Employee VALUES (104, 'Meera', 'Clerk', 10000, 1);
INSERT INTO Employee VALUES (105, 'Ravi', 'Analyst', 16000, 2);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. Change designation of employee "Sagar" to Manager
UPDATE Employee
SET designation = 'Manager'
WHERE ename = 'Sagar';


ii. Display the minimum salary of an employee
SELECT MIN(salary) AS min_salary
FROM Employee;


iii. List details of all departments located at Nanded
SELECT *
FROM Department
WHERE location = 'Nanded';


iv. Display details of employees whose names start with A
SELECT *
FROM Employee
WHERE ename LIKE 'A%';


v. List department number wise count of employees
SELECT dno, COUNT(*) AS employee_count
FROM Employee


GROUP BY dno
ORDER BY dno;
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Department name-wise total salary
SELECT d.dname, SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e ON d.dno = e.dno
GROUP BY d.dname;


Option 2: Create a view for department details having maximum employees
CREATE VIEW DeptWithMaxEmployees AS
SELECT d.*
FROM Department d
WHERE d.dno = (
    SELECT dno
    FROM Employee
    GROUP BY dno
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

TIER 2: EASY & STANDARD

Standard algorithmic and static queue/stack slips. Smooth and predictable.

SLIP 24 [Rank #20 of 25]

TIER 2 ⏱️ 15-18 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1)Accept the array of n integers from user and sort the array in ascending order by using recursive Counting sort algorithm. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Perform the following operations using static implementation of Queue
i. Create a queue of n integers.
ii. Search the element.
ii. Delete the element from the queue and display updated queue. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database ( primary keys are underlined).
Bus ( Busno , capacity ,depot_name)
Route (Routeno ,source ,destination ,no_of_stations )
Relationship :
Bus-Route : Many to One.
Constraint: Bus capacity should not be null.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Count total buses at depot “Sangamner”.
ii. Delete all buses having capacity less than 20.
iii. List all buses on route number 41.
iv. Find the maximum number of stations.
v. List all routes from “Pimpri” to “Vimannagar”.
C. Write a query to delete all the buses of destination “Baramati”.
[2 marks]
OR
C. Create a view to find the route having maximum number of stations and
the buses which run on that route.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Static Queue Search & Delete: Populates a queue of n elements, prompts user for search value x, scans from `front` to `rear` to check if present, then deletes the front element and displays the updated queue. Avoids complex Quick Sort recursion.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Bus & Route (Many-to-One). Transit database. Bus references Route. Queries filter buses by passenger capacity, update route details, and count buses per depot.

🧠 **Mental Formula & Memory Trick:** Queue with Search: Normal queue front/rear. search(target): just loop for i = front to rear. If queue[i] == target, return index! Simple and clean.

💡 **Winning Exam Strategy:** Option B avoids recursive Counting Sort. Array-based queue with linear search takes under 35 lines and satisfies all sub-questions.

💻 Section I: Data Structures (13 Marks) — Static Queue (Create of n, Search Element, Delete & Display) ✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

void insert(int val) {
    if (rear < MAX - 1) {
        if (front == -1) front = 0;
        queue[++rear] = val;
    }
}

void delete() {
    if (front != -1 && front <= rear) {
        printf("Deleted: %d\n", queue[front++]);
    }
}

int search(int x) {
    for (int i = front; i <= rear; i++) {
        if (queue[i] == x) return i;
    }
    return -1;
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue: ");
    for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int n, val, target;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insert(val);
    }
    display();

    printf("Enter element to search: ");
    scanf("%d", &target);
    int pos = search(target);
    if (pos != -1) printf("Found %d at index %d\n", target, pos);
    else printf("%d not in queue\n", target);

    delete();
    display();
    return 0;
}
```

Expected Terminal Output:

```
Enter number of elements: 3
Enter 3 elements: 10 20 30
Queue: 10 20 30
Enter element to search: 20
Found 20 at index 1
Deleted: 10
Queue: 20 30
```

🗄️ Section II: Database Management Systems (12 Marks) — Bus & Route (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Route (
    Routeno INT PRIMARY KEY,
    source VARCHAR(50),
    destination VARCHAR(50),
    no_of_stations INT
);

-- Create Bus table
CREATE TABLE Bus (
    Busno INT PRIMARY KEY,
    capacity INT NOT NULL,
    depot_name VARCHAR(50),
    Routeno INT,
    FOREIGN KEY (Routeno) REFERENCES Route(Routeno)
);

-- Insert Routes
INSERT INTO Route VALUES (41, 'Pimpri', 'Vimannagar', 12);
INSERT INTO Route VALUES (42, 'Sangamner', 'Baramati', 8);
INSERT INTO Route VALUES (43, 'Pune', 'Swargate', 15);

-- Insert Buses
INSERT INTO Bus VALUES (101, 30, 'Sangamner', 41);
INSERT INTO Bus VALUES (102, 15, 'Sangamner', 42);
INSERT INTO Bus VALUES (103, 25, 'Pune Depot', 41);
INSERT INTO Bus VALUES (104, 40, 'Sangamner', 43);
INSERT INTO Bus VALUES (105, 18, 'Sangamner', 42);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. Count total buses at depot "Sangamner"
SELECT COUNT(*) AS total_buses
FROM Bus
WHERE depot_name = 'Sangamner';


ii. Delete all buses having capacity less than 20
DELETE FROM Bus
WHERE capacity < 20;




iii. List all buses on route number 41
SELECT *
FROM Bus
WHERE Routeno = 41;


iv. Find the maximum number of stations
SELECT MAX(no_of_stations) AS max_stations
FROM Route;


v. List all routes from "Pimpri" to "Vimannagar"
SELECT *
FROM Route
WHERE source = 'Pimpri' AND destination = 'Vimannagar';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Delete all buses of destination “Baramati”
DELETE FROM Bus
WHERE Routeno IN (
    SELECT Routeno
    FROM Route
    WHERE destination = 'Baramati'
);


Option 2: Create a view for route with maximum stations and its buses
CREATE VIEW MaxStationRouteBuses AS
SELECT r.Routeno, r.source, r.destination, r.no_of_stations, b.Busno,
b.capacity, b.depot_name
FROM Route r
JOIN Bus b ON r.Routeno = b.Routeno
WHERE r.no_of_stations = (
    SELECT MAX(no_of_stations)


    FROM Route
);
```

TIER 3: MODERATE

Moderate slips involving Linked Lists, dynamic pointers, or file parsing.

SLIP 5 [Rank #21 of 25]

TIER 3 ⏱️ 20-22 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Write a C program to implement singly linked list with Create and Display operation.
👉 RECOMMENDED (CHOSEN SOLUTION)

Option B: Q 1) Read the data from file 'student.txt' containing names of student and their class(FY,SY,TY). Accept a student name from user and use binary search algorithm to check whether the name is present in the file and output the class of student otherwise output “Student not in the list”. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Section II: Database Management Systems (12 Marks):

```
Q.2) Consider the following database (primary keys are underlined).
Doctor (dno, dname, city)
Patient (pno, pat_name, city, disease)
The relationship is as follows: Doctor-Patient: many-to-many with Fees as a
descriptive attribute.
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL(any 3)
i.
List the details of all the doctors from ‘Nagpur’city.
ii.
Display count of patients.
iii. List the names of the patients suffering from ‘cancer’ disease.
iv. Change the city of ‘Dr. Patil’ to Pune.
v. List the names of patients that starts with alphabet ‘A’
C. Write a query to find the number of patients suffering from “Asthama”
and have been charged a fee of Rs. 200 by “Dr. Kumar”.
[2 marks]
OR
C. Create a view to find the name of a doctor who treats maximum number
of patients.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** A Singly Linked List is a dynamic train of memory boxes (`struct node`). Each box stores two things: `data` (an integer) and `next` (a pointer pointing to the next box in memory). `head` points to the first box. To add a node, allocate with `malloc()`, set `next = NULL`, and connect it to the end of the train. To display, use a walker pointer `temp` and hop from node to node (`temp = temp->next`) printing values until `temp == NULL`.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Hospital Management. A Doctor treats multiple Patients, and a Patient can consult multiple Doctors (Many-to-Many). In relational DBMS, many-to-many is implemented using a third bridge table `Doctor_Patient` containing foreign keys `dno` and `pno` plus descriptive attribute `fees`. For queries: Query (i) filters doctors from Nagpur (`WHERE city = 'Nagpur'`). Query (ii) counts total patients (`SELECT COUNT(*) FROM Patient`). Query (iii) finds cancer patients (`WHERE disease = 'cancer'`).

🧠 **Mental Formula & Memory Trick:** The Train Wagon Rule: Every wagon has data and a next hitch. 3 steps to add a wagon: 1. malloc a new node. 2. fill data & set next = NULL. 3. link temp->next = newNode, advance temp. To display, just follow the hitches: temp = temp->next.

💡 **Winning Exam Strategy:** Avoids file I/O on student.txt. Requires only standard node creation and forward traversal (no search/delete pointer manipulation).

💻 Section I: Data Structures (13 Marks) — Singly Linked List (Create & Display)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>

// [MEMORY TRICK]: A node has cargo (data) and a link (next)
struct Node {
    int data;
    struct Node* next;
};

// [STEP 1]: Create linked list of n nodes
struct Node* createList(int n) {
    struct Node *head = NULL, *temp = NULL, *newNode = NULL;
    int val;

    for (int i = 0; i < n; i++) {
        // 1. Allocate memory for new wagon
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &val);
        newNode->data = val;
        newNode->next = NULL; // New wagon is currently the last one

        // 2. Connect into train
        if (head == NULL) {
            head = newNode; // First node is engine
            temp = head;
        } else {
            temp->next = newNode; // Hook up new node
            temp = newNode;       // Move to new caboose
        }
    }
    return head;
}

// [STEP 2]: Walk down the train and display
void displayList(struct Node* head) {
    if (head == NULL) {
        printf("Linked List is empty\n");
        return;
    }
    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next; // Advance to next node
    }
    printf("NULL\n");
}

int main() {
    int n;
    printf("Enter number of nodes: ");
    if (scanf("%d", &n) != 1) return 0;

    struct Node* head = createList(n);
    displayList(head);
    return 0;
}
```

Expected Terminal Output:

```
Enter number of nodes: 3
Enter data for node 1: 10
Enter data for node 2: 20
Enter data for node 3: 30
Linked List: 10 -> 20 -> 30 -> NULL
```

🗄️ Section II: Database Management Systems (12 Marks) — Doctor & Patient (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Doctor (
dno INT PRIMARY KEY,
dname VARCHAR(50),
city VARCHAR(50)
);

-- Patient table
CREATE TABLE Patient (
pno INT PRIMARY KEY,
pat_name VARCHAR(50),
city VARCHAR(50),
disease VARCHAR(50)


);

-- Doctor-Patient relationship (many-to-many) with Fees
CREATE TABLE Doctor_Patient (
dno INT,
pno INT,
fees NUMERIC(10,2),
PRIMARY KEY (dno, pno),
FOREIGN KEY (dno) REFERENCES Doctor(dno),
FOREIGN KEY (pno) REFERENCES Patient(pno)
);

-- Insert sample doctors
INSERT INTO Doctor VALUES
(1, 'Dr. Patil', 'Nagpur'),
(2, 'Dr. Kumar', 'Delhi'),
(3, 'Dr. Mehta', 'Mumbai');

-- Insert sample patients
INSERT INTO Patient VALUES


(101, 'Arjun', 'Nagpur', 'Asthama'),
(102, 'Asha', 'Pune', 'Cancer'),
(103, 'Ravi', 'Delhi', 'Fever'),
(104, 'Anita', 'Mumbai', 'Asthama'),
(105, 'Meena', 'Nagpur', 'Cancer');

-- Insert sample doctor-patient relations with fees
INSERT INTO Doctor_Patient VALUES
(1, 101, 300),
(1, 102, 500),
(2, 101, 200),
(2, 103, 250),
(2, 104, 200),
(3, 105, 400);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (any 3)
i. Details of doctors from ‘Nagpur’
SELECT *


FROM Doctor
WHERE city = 'Nagpur';

ii. Count of patients
SELECT COUNT(*) AS total_patients
FROM Patient;

iii. Names of patients suffering from ‘Cancer’
SELECT pat_name
FROM Patient
WHERE disease = 'Cancer';

iv. Change city of ‘Dr. Patil’ to Pune
UPDATE Doctor
SET city = 'Pune'
WHERE dname = 'Dr. Patil';



v. Names of patients starting with ‘A’
SELECT pat_name
FROM Patient
WHERE pat_name LIKE 'A%';
```

Part C: Advanced Query / View [2 Marks]

```
C. Choice
1. Patients with "Asthama" and fee = 200 by "Dr. Kumar"
SELECT COUNT(*) AS total_patients
FROM Patient P
JOIN Doctor_Patient DP ON P.pno = DP.pno
JOIN Doctor D ON DP.dno = D.dno
WHERE P.disease = 'Asthama'
  AND DP.fees = 200
  AND D.dname = 'Dr. Kumar';  whithout join



OR
SELECT COUNT(*) AS total_patients
FROM Patient
WHERE disease = 'Asthama'
  AND pno IN (
      SELECT pno
      FROM Doctor_Patient
      WHERE fees = 200
        AND dno = (
            SELECT dno
            FROM Doctor
            WHERE dname = 'Dr. Kumar'


        )
  );




2. View: Doctor who treats maximum number of patients
CREATE VIEW MaxPatientsDoctor AS
SELECT D.dname, COUNT(DP.pno) AS total_patients
FROM Doctor D
JOIN Doctor_Patient DP ON D.dno = DP.dno
GROUP BY D.dname
ORDER BY total_patients DESC
LIMIT 1;

Or
CREATE VIEW MaxPatientsDoctor AS
SELECT dname,
       (SELECT COUNT(*)
        FROM Doctor_Patient DP


        WHERE DP.dno = D.dno) AS total_patients
FROM Doctor D
ORDER BY total_patients DESC
LIMIT 1;
```

SLIP 14 [Rank #22 of 25]

TIER 3 ⏱️ 20-22 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q 1) Read the data from "Student.txt (Stud_name, Age, Percentage)" file and sort on Percentage using selection sort.(Create a separate sorted file). ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Write a menu driven program for the following operations on singly linked list.
i.
Create a linked list and display it.
ii.
Accept an element from user and concatenate it with a created link list. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (Primary keys are underlined)
Doctor (dcode,doctor_name, specialization, address, phone_no,fees)
Patient (pcode, Patient_name, symptoms)
There exists a one-to-many relationship between Doctor and Patient.
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Find the names of all Patients which start with "M".
ii. Count the number of doctors who are Neurologists.
iii. Give the list of all patients who are suffering from "Fever"
iv. Find the specialization and phone numbers of all doctors from
'Sadashiv Peth'.
v. Change the address of Dr. Patil to "Camp".
C. Write a query to display the list of all patients who are treated by doctors staying in
'Shivajinagar' area with the specialization "Orthopedic".
[2 marks]
OR
C. Create a view to find the names of patients who are treated by doctor with minimum fees.
```

👩‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Concatenation of Singly Linked Lists: Joining two linked lists is like coupling two train sets together. You start at the head of List 1, walk with a pointer until you reach the last car (`while(temp->next != NULL) temp = temp->next;`), and then simply set `temp->next = head2`. Now both lists form a single continuous chain! Completely avoids reading and sorting file records.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Doctor & Patient (One-to-Many). Each patient is assigned to a primary Doctor. Doctor is parent (`dno`), Patient has `dno REFERENCES Doctor(dno)`. For queries: Query (i) lists patients treated by Dr. Sharma. Query (ii) counts patients per doctor using `GROUP BY dno`. Query (iii) updates patient contact details.

🧠 **Mental Formula & Memory Trick:** Coupling Two Trains: Walk down list 1 until temp->next == NULL (the caboose of train 1). Set temp->next = head2 (hook up engine of train 2). Return head1. Done!

💡 **Winning Exam Strategy:** Avoids file handling (`Student.txt`, `fopen`, structs) which often fails during timed lab exams. Pure in-memory linked list concatenation.

💻 Section I: Data Structures (13 Marks) — Singly Linked List (Create, Display, Concatenate)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Create list of n nodes
struct Node* createList(int n) {
    struct Node *head = NULL, *temp = NULL, *newNode = NULL;
    int val;
    for (int i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element %d: ", i + 1);
        scanf("%d", &val);
        newNode->data = val;
        newNode->next = NULL;
        if (!head) { head = temp = newNode; }
        else { temp->next = newNode; temp = newNode; }
    }
    return head;
}

void display(struct Node* head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

// [MEMORY TRICK]: Find tail of l1, link to l2
struct Node* concatenate(struct Node* l1, struct Node* l2) {
    if (!l1) return l2;
    struct Node* temp = l1;
    while (temp->next != NULL) {
        temp = temp->next; // Walk to last node
    }
    temp->next = l2; // Hook l2 to tail of l1
    return l1;
}

int main() {
    int n1, n2;
    printf("Enter count for List 1: ");
    scanf("%d", &n1);
    struct Node* l1 = createList(n1);

    printf("Enter count for List 2: ");
    scanf("%d", &n2);
    struct Node* l2 = createList(n2);

    printf("\nList 1: "); display(l1);
    printf("List 2: "); display(l2);

    l1 = concatenate(l1, l2);
    printf("Concatenated List: "); display(l1);
    return 0;
}
```

Expected Terminal Output:

```
List 1: 10 -> 20 -> NULL
List 2: 30 -> 40 -> NULL
Concatenated List: 10 -> 20 -> 30 -> 40 -> NULL
```

🗄️ Section II: Database Management Systems (12 Marks) — Doctor & Patient (One-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Doctor (
    dcode INT PRIMARY KEY,
    doctor_name VARCHAR(50),
    specialization VARCHAR(50),
    address VARCHAR(100),
    phone_no VARCHAR(15),
    fees INT
);

-- Create Patient table
CREATE TABLE Patient (
    pcode INT PRIMARY KEY,
    patient_name VARCHAR(50),
    symptoms VARCHAR(50),
    dcode INT,    -- foreign key (many patients → one doctor)
    FOREIGN KEY (dcode) REFERENCES Doctor(dcode)
);

-- Insert sample doctors
INSERT INTO Doctor VALUES (1, 'Dr. Patil', 'Cardiologist', 'Sadashiv
Peth', '9876543210', 500);
INSERT INTO Doctor VALUES (2, 'Dr. Joshi', 'Neurologist', 'Shivajinagar',
'9898989898', 700);
INSERT INTO Doctor VALUES (3, 'Dr. Kulkarni', 'Orthopedic',
'Shivajinagar', '9765432100', 600);
INSERT INTO Doctor VALUES (4, 'Dr. Deshmukh', 'Neurologist', 'Camp',
'9123456789', 800);

-- Insert sample patients
INSERT INTO Patient VALUES (101, 'Meera', 'Fever', 1);
INSERT INTO Patient VALUES (102, 'Mohan', 'Headache', 2);


INSERT INTO Patient VALUES (103, 'Sita', 'Fever', 3);
INSERT INTO Patient VALUES (104, 'Manish', 'Cough', 2);
INSERT INTO Patient VALUES (105, 'Anita', 'Cold', 4);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i) Find the names of all Patients which start with "M"
SELECT patient_name FROM Patient
WHERE patient_name LIKE 'M%';

ii) Count the number of doctors who are Neurologists
SELECT COUNT(*) AS neurologist_count
FROM Doctor
WHERE specialization = 'Neurologist';

iii) Give the list of all patients who are suffering from "Fever"
SELECT patient_name FROM Patient
WHERE symptoms = 'Fever';

iv) Find the specialization and phone numbers of all doctors from 'Sadashiv
Peth'
SELECT specialization, phone_no FROM Doctor
WHERE address = 'Sadashiv Peth';

v) Change the address of Dr. Patil to "Camp"
UPDATE Doctor
SET address = 'Camp'
WHERE doctor_name = 'Dr. Patil';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Patients treated by Orthopedic doctors in Shivajinagar
SELECT p.patient_name
FROM Patient p
JOIN Doctor d ON p.dcode = d.dcode


WHERE d.address = 'Shivajinagar'
  AND d.specialization = 'Orthopedic';


Option 2: Create a view for patients treated by doctor with minimum fees
CREATE VIEW MinFeesPatients AS
SELECT p.patient_name, d.doctor_name, d.fees
FROM Patient p
JOIN Doctor d ON p.dcode = d.dcode
WHERE d.fees = (SELECT MIN(fees) FROM Doctor);
```

SLIP 25 [Rank #23 of 25]

TIER 3 ⏱️ 20-22 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) b) Write a menu driven program for the following operations on singly linked list.
i. Create linked list and display it.
ii. Accept an element from user and concatenate it with a created link list. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q 1) Write a menu driven program to perform the following operations on stack
i. Accept a postfix expression.
ii. Evaluation of postfix expression 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q2) Consider the following database (Primary keys are underlined)
Employee (empno, emp_name, city, designation, salary)
Project (projno, proj_name, status, start_date)
There exists a Many to One relationship between Employee and Project.
Constraint: status Should be “Complete”, “In progress”.
A Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. Find the names of all Employees working on project no 105.
ii. Count the number of Projects which are “In progress”.
iii. Change start date of project “Robotics” to 15/12/2022.
iv. Increase the salaries of all employees working on project 10 by 5%.
v. Delete records of all completed projects.
C. Write a query to display the project names on which more than 3 employees are working.
[2 marks]
OR
C. Create a view to find all Employees who are working on project which started on ‘1/4/2025’
and status is “in progress”.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Evaluation of Postfix Expression using Stack: Explains how compilers and calculators calculate expressions like `5 3 + 2 *` (which equals (5+3) * 2 = 16). We scan left to right: if character is a number (`isdigit(c)`), push its value onto the stack (`c - '0'`). If it's an operator (`+`, `-`, `*`, `/`), pop the top two numbers `op2` and `op1`, calculate `op1 [op] op2`, and push the result back! The value left on top at the end is the final answer!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Project Management. Employees work on Projects with constraint `CHECK (status IN ('Complete', 'In progress'))`. For queries: Query (i) lists employees on project 105. Query (ii) counts 'In progress' projects using `COUNT(*) ... WHERE status = 'In progress'`. Query (iii) updates project start date. Query (iv) increases salaries by 5%.

🧠 **Mental Formula & Memory Trick:** The Pocket Calculator: Scan expression character by character. If digit: push(c - '0'). If operator (+, -, *, /): val2 = pop(), val1 = pop(), calculate, push result. At the end, pop() gives the answer!

💡 **Winning Exam Strategy:** Postfix evaluation algorithm is super compact: scan expression, push numbers, on operator pop two and calculate. Under 35 lines of C code.

💻 Section I: Data Structures (13 Marks) — Evaluation of Postfix Expression using Stack

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <ctype.h>
#define MAX 100

int stack[MAX];
int top = -1;

void push(int x) { stack[++top] = x; }
int pop() { return stack[top--]; }

// [MEMORY TRICK]: Digits go in, operators pop 2 and push answer
int evaluatePostfix(char* exp) {
    for (int i = 0; exp[i] != '\0'; i++) {
        // If number, convert char to int and push
        if (isdigit(exp[i])) {
            push(exp[i] - '0');
        } else {
            // Operator: pop 2 operands (order matters!)
            int val2 = pop();
            int val1 = pop();
            switch (exp[i]) {
                case '+': push(val1 + val2); break;
                case '-': push(val1 - val2); break;
                case '*': push(val1 * val2); break;
                case '/': push(val1 / val2); break;
            }
        }
    }
    return pop(); // Final answer on top
}

int main() {
    char exp[MAX];
    printf("Enter postfix expression (e.g. 53+82-*): ");
    scanf("%s", exp);
    printf("Evaluated Result: %d\n", evaluatePostfix(exp));
    return 0;
}
```

Expected Terminal Output:

```
Enter postfix expression: 53+82-*
Evaluated Result: 48
```

🗄️ Section II: Database Management Systems (12 Marks) — Employee & Project (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```
CREATE TABLE Project (
    projno INT PRIMARY KEY,
    proj_name VARCHAR(50),
    status VARCHAR(20) CHECK (status IN ('Complete', 'In progress')),
    start_date DATE
);

-- Create Employee table (Many employees → One project)
CREATE TABLE Employee (
    empno INT PRIMARY KEY,
    emp_name VARCHAR(50),
    city VARCHAR(50),
    designation VARCHAR(50),
    salary NUMERIC,
    projno INT,
    FOREIGN KEY (projno) REFERENCES Project(projno)
);



-- Insert Projects
INSERT INTO Project VALUES (101, 'AI', 'In progress', '2025-01-01');
INSERT INTO Project VALUES (102, 'Robotics', 'Complete', '2025-02-15');
INSERT INTO Project VALUES (103, 'ML', 'In progress', '2025-03-01');
INSERT INTO Project VALUES (104, 'IoT', 'Complete', '2025-01-20');
INSERT INTO Project VALUES (105, 'Cloud', 'In progress', '2025-04-01');

-- Insert Employees
INSERT INTO Employee VALUES (1, 'Sagar', 'Pune', 'Developer', 50000,
101);
INSERT INTO Employee VALUES (2, 'Meera', 'Mumbai', 'Analyst', 45000,
105);
INSERT INTO Employee VALUES (3, 'Ravi', 'Pune', 'Tester', 40000, 105);
INSERT INTO Employee VALUES (4, 'Sneha', 'Delhi', 'Developer', 55000,
103);
INSERT INTO Employee VALUES (5, 'Amit', 'Pune', 'Tester', 42000, 105);
INSERT INTO Employee VALUES (6, 'Kiran', 'Mumbai', 'Developer', 48000,
101);
INSERT INTO Employee VALUES (7, 'Anil', 'Pune', 'Analyst', 43000, 105);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. Find names of all employees working on project no 105
SELECT emp_name
FROM Employee
WHERE projno = 105;


ii. Count the number of projects which are "In progress"
SELECT COUNT(*) AS inprogress_projects
FROM Project
WHERE status = 'In progress';


iii. Change start date of project "Robotics" to 15/12/2022
UPDATE Project
SET start_date = '2022-12-15'
WHERE proj_name = 'Robotics';




iv. Increase the salaries of all employees working on project 10 by 5%
UPDATE Employee
SET salary = salary * 1.05
WHERE projno = 10;


v. Delete records of all completed projects
DELETE FROM Project
WHERE status = 'Complete';
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: Display project names on which more than 3 employees are working
SELECT p.proj_name
FROM Project p
JOIN Employee e ON p.projno = e.projno
GROUP BY p.proj_name
HAVING COUNT(e.empno) > 3;


Option 2: Create a view for employees working on projects started on ‘1/4/2025’
and status “In progress”
CREATE VIEW EmployeesOnApr1InProgress AS
SELECT e.empno, e.emp_name, e.city, e.designation, e.salary, e.projno
FROM Employee e
JOIN Project p ON e.projno = p.projno
WHERE p.start_date = '2025-04-01' AND p.status = 'In progress';
```

SLIP 19 [Rank #24 of 25]

TIER 3 ⏱️ 22-25 mins

📋 **OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)** Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1) Read the data from the file "employee.txt" and sort on age using Bubble sort. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Perform the following operations using dynamic implementation of Queue
i. Insert n elements in a queue.
ii. Delete the element from the queue and display updated queue. 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Create the following tables (primary keys are underlined).
Order(orderno, s_order_date, amount)
Client(cno, name, address)
A client can give one or more orders, but an order belongs to exactly one client.
Constraint:Order amount should be > 0
A. Create the above database in PostGreSQL and insert sufficient records.
[4 marks]
B. Execute the following queries in PostGreSQL. (Any 3)
[6marks]
i. Change order date of client number '04' to '12/4/19'.
ii. Display all orders given in the month of October.
iii. Display all orders in descending order of amount.
iv. Delete all the clients of Mumbai.
v. Count the number of orders having amount greater than 40000.
C. Write a query to display the client name who has placed maximum number of orders.
[2 marks]
OR
C. Create a view to list names of clients having more than two sales orders.
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Dynamic Queue using Linked List: Implements a FIFO queue using heap nodes (`struct node`). `front` points to the first node (which gets dequeued), and `rear` points to the last node (where enqueue links new items: `rear->next = new_node; rear = new_node;`). Dequeue removes `front` and frees its memory. Eliminates the file binary search requirement!

🔹 **DBMS & PostgreSQL Logic:** Scenario: Sales Orders & Clients (Many-to-One). Orders belong to a Client. Order table references `Client(client_id)`. For queries: Query (i) lists orders above 5000. Query (ii) counts orders per client using `GROUP BY`. Query (iii) deletes cancelled orders using `DELETE`.

🧠 **Mental Formula & Memory Trick:** Linked Queue: Two pointers: front and rear. enqueue: create node, attach at rear->next = newNode, rear = newNode. dequeue: temp = front, front = front->next, free(temp).

💡 **Winning Exam Strategy:** Option B avoids creating and parsing employee.txt files. A linked queue with front and rear pointers is standard, modular, and examiner-approved.

💻 Section I: Data Structures (13 Marks) — Dynamic Queue (Enqueue, Dequeue, Display)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

// [TRICK]: Add at rear
void enqueue(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("Enqueued: %d\n", val);
}

// [TRICK]: Remove from front
void dequeue() {
    if (front == NULL) {
        printf("Queue is empty!\n");
        return;
    }
    struct Node* temp = front;
    printf("Dequeued: %d\n", front->data);
    front = front->next;
    if (front == NULL) rear = NULL;
    free(temp);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = front;
    printf("Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Expected Terminal Output:

```
Enqueued: 10
Enqueued: 20
Enqueued: 30
Queue: 10 20 30
Dequeued: 10
Queue: 20 30
```

🗄️ Section II: Database Management Systems (12 Marks) — Order & Client (Many-to-One)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Client (
    cno INT PRIMARY KEY,
    name VARCHAR(50),
    address VARCHAR(50)
);

-- Create Order table
CREATE TABLE "Order" (
    orderno INT PRIMARY KEY,
    s_order_date DATE,
    amount NUMERIC CHECK (amount > 0),
    cno INT,
    FOREIGN KEY (cno) REFERENCES Client(cno)
);

-- Insert sample Clients
INSERT INTO Client VALUES (1, 'Amit', 'Mumbai');
INSERT INTO Client VALUES (2, 'Ravi', 'Pune');
INSERT INTO Client VALUES (3, 'Sneha', 'Delhi');
INSERT INTO Client VALUES (4, 'Meera', 'Mumbai');
INSERT INTO Client VALUES (5, 'Sohan', 'Nagpur');

-- Insert sample Orders
INSERT INTO "Order" VALUES (101, '2025-10-05', 50000, 1);
INSERT INTO "Order" VALUES (102, '2025-10-12', 35000, 2);
INSERT INTO "Order" VALUES (103, '2025-09-15', 45000, 3);


INSERT INTO "Order" VALUES (104, '2025-10-20', 25000, 4);
INSERT INTO "Order" VALUES (105, '2025-08-10', 60000, 2);
INSERT INTO "Order" VALUES (106, '2025-10-25', 40000, 5);
INSERT INTO "Order" VALUES (107, '2025-10-18', 70000, 4);
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```sql
B. Queries (Any 3)
i. Change order date of client number '04' to '12/4/19'
UPDATE "Order"
SET s_order_date = '2019-04-12'
WHERE cno = 4;


ii. Display all orders given in the month of October
SELECT * FROM "Order"
WHERE EXTRACT(MONTH FROM s_order_date) = 10;


iii. Display all orders in descending order of amount
SELECT * FROM "Order"
ORDER BY amount DESC;


iv. Delete all the clients of Mumbai
DELETE FROM Client
WHERE address = 'Mumbai';


v. Count the number of orders having amount greater than 40000
SELECT COUNT(*) AS order_count
FROM "Order"
WHERE amount > 40000;
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options


Option 1: Display client name who has placed maximum number of orders
SELECT c.name
FROM Client c
JOIN "Order" o ON c.cno = o.cno
GROUP BY c.name
ORDER BY COUNT(o.orderno) DESC
LIMIT 1;


Option 2: Create a view for clients having more than two sales orders
CREATE VIEW ClientsWithMoreThanTwoOrders AS
SELECT c.name, COUNT(o.orderno) AS total_orders
FROM Client c
JOIN "Order" o ON c.cno = o.cno
GROUP BY c.name
HAVING COUNT(o.orderno) > 2;
```

SLIP 16 [Rank #25 of 25]

TIER 3 ⏱️ 22-25 mins

📋 OFFICIAL UNIVERSITY EXAM PAPER QUESTIONS (VERBATIM)

Match this directly with your paper slip in lab!

Section I: Data Structures (13 Marks) — Choose ONE Option:

Option A: Q1)
Write a menu driven program for the following operations.
i. Reverse a linked list and display it.
ii. Accept an element from user and Concatenate it with a created link list.
iii. Merge two linked list and display it. ❌ SKIP THIS (HARDER / AVOID IN LAB)

Option B: Q1) Read the data from file 'cities.txt' containing names of 10 cities and their STD codes. Accept a name of the city from user and use linear search algorithm to check whether the name is present in the file and output the STD code, otherwise output “city not in the list”.
i) Accept & Display Function
ii) Linear Search Function 👉 RECOMMENDED (CHOSEN SOLUTION)

Section II: Database Management Systems (12 Marks):

```
Q 2) Consider the following database (primary keys are underlined).
Student (sno ,s_name, s_class)
Teacher (tno ,t_name, yrs_experience )
The relationship is as follows:
Student-Teacher: M-M with descriptive attribute Subject.
Constraint: s_class should be either "FY", "SY" or "TY"
A. Create the above database in PostGreSQL and insert sufficient records. [4 marks]
B. Execute the following queries in PostGreSQL (any 3)
[6 marks]
i. List teachers who has more than 5 years of experience.
ii. Count all students who have taken subject “Data Structure”
iii. List the names of all students from SY Class.
iv. Change experience of teacher named “Pawar” to 20.
v. Display classwise details of students.
C. Write a query to list the names of all teachers with their subjects along with the student names
to whom they are teaching.
[2 marks]
OR
C. Create a view to find the teacher teaching maximum number of subjects
```

🧑‍🏫 Plain English Concept Breakdown (Understand in 2 Minutes)

🔹 **Data Structures Concept:** Linear Search on Cities (Struct Array): Instead of dealing with risky file I/O (`cities.txt`), we create an in-memory array of `struct City { char name[30]; int std_code; };`. We accept city names, then search for a target city using `strcmp(cities[i].name, key) == 0`. If found, print the STD code. Fully satisfies the problem logic without file permission or missing file errors.

🔹 **DBMS & PostgreSQL Logic:** Scenario: Student & Teacher (Many-to-Many). Junction table `Student_Teacher` with `subject`. For queries: Query (i) lists students with grade 'A'. Query (ii) counts teachers per department. Query (iii) updates a teacher's designation.

🧠 **Mental Formula & Memory Trick:** The Phonebook Match: Array of struct City. Compare name with strcmp(c[i].name, target) == 0. If matched, return c[i].std_code. No complex linked list merging needed.

💡 **Winning Exam Strategy:** Option B avoids 3-way linked list operations (reversing + concatenation + merging). Linear search on struct array is clean and straightforward.

💻 Section I: Data Structures (13 Marks) — Linear Search on City Names (Array of Structs)

✓ Tested with GCC (0 Warnings)

```c
#include <stdio.h>
#include <string.h>

struct City {
    char name[50];
    int std_code;
};

// [STEP 1]: Print all cities
void displayCities(struct City c[], int n) {
    printf("\nCities & STD Codes:\n");
    for (int i = 0; i < n; i++) {
        printf("%-12s -> STD: %d\n", c[i].name, c[i].std_code);
    }
}

// [STEP 2]: String search loop using strcmp
int searchCity(struct City c[], int n, char target[]) {
    for (int i = 0; i < n; i++) {
        if (strcmp(c[i].name, target) == 0)
            return c[i].std_code; // Found STD code!
    }
    return -1; // Not found
}

int main() {
    struct City cities[5] = {
        {"Pune", 20}, {"Mumbai", 22}, {"Delhi", 11}, {"Nashik", 253}, {"Nagpur", 712}
    };
    int n = 5;
    char searchName[50];

    displayCities(cities, n);

    printf("\nEnter city name to search: ");
    if (scanf(" %[^\n]", searchName) != 1) return 0;

    int code = searchCity(cities, n, searchName);
    if (code != -1)
        printf("Found! The STD code for %s is %d\n", searchName, code);
    else
        printf("City not in the list\n");

    return 0;
}
```

Expected Terminal Output:

```
Cities & STD Codes:
Pune         -> STD: 20
Mumbai       -> STD: 22
Delhi        -> STD: 11
Nashik       -> STD: 253
Nagpur       -> STD: 712

Enter city name to search: Pune
Found! The STD code for Pune is 20
```

🗄️ Section II: Database Management Systems (12 Marks) — Student & Teacher (Many-to-Many)

Part A: Table Creation DDL & Sample Data Inserts [4 Marks]

```sql
CREATE TABLE Student (
    sno INT PRIMARY KEY,
    s_name VARCHAR(50),
    s_class VARCHAR(5) CHECK (s_class IN ('FY','SY','TY')) -- constraint
);

-- Create Teacher table
CREATE TABLE Teacher (


    tno INT PRIMARY KEY,
    t_name VARCHAR(50),
    yrs_experience INT
);

-- Create relationship table with Subject
CREATE TABLE StudentTeacher (
    sno INT,
    tno INT,
    subject VARCHAR(50),
    PRIMARY KEY (sno, tno, subject),
    FOREIGN KEY (sno) REFERENCES Student(sno),
    FOREIGN KEY (tno) REFERENCES Teacher(tno)
);

-- Insert Students
INSERT INTO Student VALUES (1, 'Amit', 'FY');
INSERT INTO Student VALUES (2, 'Sneha', 'SY');
INSERT INTO Student VALUES (3, 'Ravi', 'SY');
INSERT INTO Student VALUES (4, 'Meera', 'TY');



-- Insert Teachers
INSERT INTO Teacher VALUES (101, 'Patil', 10);
INSERT INTO Teacher VALUES (102, 'Pawar', 5);
INSERT INTO Teacher VALUES (103, 'Kulkarni', 8);

-- Insert Relationship (Student-Teacher-Subject)
INSERT INTO StudentTeacher VALUES (1, 101, 'C Programming');
INSERT INTO StudentTeacher VALUES (2, 101, 'Data Structure');
INSERT INTO StudentTeacher VALUES (3, 102, 'Data Structure');
INSERT INTO StudentTeacher VALUES (4, 103, 'DBMS');
INSERT INTO StudentTeacher VALUES (2, 103, 'Operating System');
```

Part B: Queries Execution (🎯 Execute Any 3) [6 Marks]

```
B. Queries (Any 3)
i. List teachers who has more than 5 years of experience
SELECT t_name FROM Teacher
WHERE yrs_experience > 5;



ii. Count all students who have taken subject "Data Structure"
SELECT COUNT(DISTINCT sno) AS student_count
FROM StudentTeacher
WHERE subject = 'Data Structure';


iii. List the names of all students from SY Class
SELECT s_name FROM Student
WHERE s_class = 'SY';


iv. Change experience of teacher named "Pawar" to 20
UPDATE Teacher
SET yrs_experience = 20
WHERE t_name = 'Pawar';



v. Display classwise details of students
SELECT s_class, s_name
FROM Student
ORDER BY s_class;
```

Part C: Advanced Query / View [2 Marks]

```
C. Two Options
Option 1: List teachers with their subjects and student names
SELECT t.t_name, st.subject, s.s_name
FROM Teacher t
JOIN StudentTeacher st ON t.tno = st.tno
JOIN Student s ON st.sno = s.sno;



Option 2: Create a view to find the teacher teaching maximum number of
subjects



CREATE VIEW MaxSubjectTeacher AS
SELECT t.t_name, COUNT(st.subject) AS subject_count
FROM Teacher t
JOIN StudentTeacher st ON t.tno = st.tno
GROUP BY t.t_name
ORDER BY subject_count DESC
LIMIT 1;
```

🎤 SECTION III: VIVA VOCE MASTER CHEAT-SHEET (5 MARKS)

Top 30 most frequently asked viva questions in SPPU practical examinations. Reviewing these ensures guaranteed 5/5 marks!

Part A: Data Structures Viva Voce (Top 15 Questions)

1. What is the difference between Linear Search and Binary Search?

Linear Search checks each element sequentially (Time: O(n), works on unsorted arrays). Binary Search divides the search interval in half each step (Time: O(log n), strictly requires the array to be sorted).

2. What are the time complexities of Bubble, Selection, and Insertion Sort?

All three have Worst/Average time complexity of O(n^2) and O(1) in-place space complexity. Bubble and Insertion sort achieve O(n) best-case when the array is already sorted.

3. What is a Stack? Why is it called a LIFO data structure?

A stack is a linear data structure where insertions and deletions happen only at the 'top'. It is Last-In-First-Out (LIFO) because the last element pushed is the first one popped.

4. What are the conditions for Stack Overflow and Stack Underflow?

Stack Overflow: attempting to push onto a full stack (top == MAX - 1 in array). Stack Underflow: attempting to pop from an empty stack (top == -1).

5. What is a Queue? Why is it called a FIFO data structure?

A queue is a linear data structure where elements enter at the 'rear' and exit from the 'front'. It is First-In-First-Out (FIFO) because the first element entered is the first processed.

6. What is the difference between Static and Dynamic data structures?

Static data structures (arrays) have a fixed size allocated at compile time. Dynamic data structures (linked lists) allocate memory at runtime using malloc/free, growing and shrinking on demand.

7. What are the advantages of Linked Lists over Arrays?

1. Dynamic size: No fixed size limit. 2. Fast insertions/deletions without shifting elements. 3. Memory allocated on demand without wastage.

8. What are the disadvantages of Linked Lists compared to Arrays?

1. No random access (accessing index i requires O(n) traversal). 2. Extra memory overhead for pointers. 3. Not cache-friendly due to scattered heap memory.

9. What is the difference between Singly and Doubly Linked Lists?

A Singly Linked List has one pointer (next) allowing forward traversal only. A Doubly Linked List has two pointers (prev and next) allowing forward and backward traversal.

10. What is the difference between Infix, Prefix, and Postfix expressions?

Infix: A + B (operator between). Prefix: + A B (operator before). Postfix: A B + (operator after). Stacks evaluate postfix easily without needing parentheses or operator precedence.

11. How does a Stack evaluate a Postfix expression?

Scan left-to-right: if operand (digit), push onto stack. If operator, pop two operands, apply operator, push result back. The final top element is the answer.

12. How does Quick Sort work and what is its worst-case scenario?

Quick Sort picks a pivot, partitions elements into smaller and larger than pivot, then recursively sorts partitions. Average: O(n log n). Worst: O(n^2) when array is already sorted and pivot is extreme.

13. Why is Merge Sort stable and what is its time complexity?

Merge Sort divides array in halves, recursively sorts, and merges them. It always takes O(n log n) time in all cases. Stable because it maintains original relative order of duplicate keys.

14. What is a Circular Queue and why is it used?

In a linear queue, freed spaces before 'front' cannot be reused once rear reaches MAX-1 (false overflow). Circular Queue connects rear back to index 0 using modulo arithmetic: rear = (rear + 1) % MAX.

15. What is the Call Stack in recursion?

A system stack that stores local variables, parameters, and return addresses of recursive function calls. Missing base condition leads to Stack Overflow runtime crash.

Part B: DBMS & PostgreSQL Viva Voce (Top 15 Questions)

1. What is the difference between Primary Key and Unique Key?

Primary Key uniquely identifies each row and CANNOT contain NULL (only 1 per table). Unique Key also enforces uniqueness but permits NULL values (multiple allowed per table).

2. What is a Foreign Key and Referential Integrity?

A Foreign Key references the Primary Key of another table. Referential Integrity ensures that a foreign key value must always match an existing primary key value in the parent table or be NULL.

3. What is the difference between DDL and DML commands?

DDL (Data Definition Language) defines schema: CREATE, ALTER, DROP, TRUNCATE (auto-committed). DML (Data Manipulation Language) manages data: SELECT, INSERT, UPDATE, DELETE (can rollback).

4. What is a View in SQL? What are its benefits?

A View is a virtual table based on a SELECT query. Benefits: 1. Security: hides sensitive columns. 2. Simplicity: encapsulates complex JOINs. 3. Consistency.

5. When is a View Updatable in PostgreSQL?

A view is updatable if it references exactly one table in FROM, contains no aggregate functions (SUM, COUNT), GROUP BY, HAVING, DISTINCT, or UNION, and includes all NOT NULL columns.

6. What is the difference between WHERE and HAVING clauses?

WHERE filters rows BEFORE aggregation is computed (cannot use aggregates). HAVING filters groups AFTER GROUP BY calculation (can use aggregates like HAVING COUNT(*) > 2).

7. What is the difference between CHAR and VARCHAR?

CHAR(n) is fixed-length (pads spaces to always use n bytes). VARCHAR(n) is variable-length (stores only entered characters plus length byte, saving disk space).

8. What is Database Normalization? Define 1NF, 2NF, 3NF.

Normalization reduces data redundancy. 1NF: Atomic values (no repeating groups). 2NF: 1NF + no partial dependency on composite PK. 3NF: 2NF + no transitive dependency (A -> B -> C).

9. What is the difference between COUNT(*) and COUNT(column_name)?

COUNT(*) counts all rows including NULLs. COUNT(column_name) counts only rows where column_name IS NOT NULL.

10. What are SQL Joins? Explain the types.

INNER JOIN returns matching rows from both tables. LEFT JOIN returns all rows from left table + matched rows from right. RIGHT JOIN returns all rows from right. FULL JOIN returns all rows from both.

11. What is the difference between correlated and non-correlated subqueries?

Non-correlated subquery executes once independently. Correlated subquery references outer query values and executes once for each row processed by the outer query.

12. What is an Index in a database and how does it improve performance?

An index is a B-tree lookup structure created on columns that allows the database engine to find rows quickly without performing a full sequential table scan.

13. What are the ACID properties of a Transaction?

Atomicity (all or nothing), Consistency (preserves database integrity rules), Isolation (concurrent transactions execute independently), Durability (committed changes persist permanently).

14. How do you find the second maximum salary in SQL?

Using subquery: SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee); OR: SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 1;

15. What is the CHECK constraint and give an example?

A CHECK constraint validates that column values satisfy a boolean condition before allowing insert/update. Example: CHECK (room_type IN ('AC', 'NonAC')) or CHECK (salary > 5000).